



Akademia
Techniczno-Humanistyczna
w Bielsku-Białej

PRACA DYPLOMOWA

WYDZIAŁ
BUDOWY MASZYN I INFORMATYKI
KIERUNEK: Informatyka
SPECJALNOŚĆ: Techniki tworzenia oprogramowania

JAN BORYCZKO

nr albumu: 54384

Praca magisterska

PROGRAM KOMPUTEROWY DO OPTYMALIZACJI
UKŁADU TOWARU W MAGAZYNIE Z WYKORZYSTANIEM
ALGORYTMU GENETYCZNEGO

kategoria pracy: praca konstrukcyjna

Promotor: dr hab. inż. MIROSŁAW KORDOS, prof. ATH

Bielsko-Biała, rok akademicki 2022/2023

Spis treści

Wykaz skrótów i oznaczeń	4
Wstęp	5
Cel pracy	6
Zakres pracy	7
1 Studium literaturowe z zakresu algorytmów genetycznych i algorytmów genetycznych wielokryterialnych	8
1.1 Podstawy algorytmów genetycznych	8
1.1.1 Optymalizacja	8
1.1.2 Definicja i działanie algorytmu genetycznego	8
1.1.3 Operatory algorytmu genetycznego	10
1.1.4 Warunek stopu	11
1.2 Podstawy algorytmów genetycznych wielokryterialnych	12
1.2.1 Optymalizacja wielokryterialna	12
1.2.2 Działanie algorytmu genetycznego wielokryterialnego	12
1.3 Analiza istniejących rozwiązań	15
1.4 Analiza porównawcza przedstawionych rozwiązań	18
2 Definicja założeń do realizacji projektu	20
2.1 Platforma programistyczna i język programowania	20
2.2 Format danych wejściowych	20
2.2.1 Lista towarów	20
2.2.2 Lista towarów powiązanych	21
2.2.3 Struktura magazynu	21
2.2.4 Struktura magazynu - macierz	22
2.2.5 Struktura magazynu - regały	22
2.3 Kodowanie osobnika	23
2.4 Operatory algorytmu genetycznego	23
2.4.1 Fitness i kryteria oceny	25
2.4.2 Operator selekcji i mutacji	26
2.4.3 Operatory krzyżowania	27
3 Projekt programu	29
3.1 Parametry wejściowe	29
3.2 Działanie programu	30
3.3 Przebieg algorytmu	30
3.4 Użyte pakiety	32

3.5	Struktura kodu	35
3.6	Elementy i operatory algorytmu	36
3.6.1	Fitness	37
3.6.2	Ranga i odległość zagęszczenia	38
3.6.3	Selekcja	38
3.6.4	Mutacja	40
3.6.5	Krzyżowanie AEX	40
3.6.6	Krzyżowanie ERX	41
3.6.7	Krzyżowanie PMX	41
3.7	Wyznaczanie rozwiązania najlepszego oraz współczynnik poprawy	43
3.8	Interfejs użytkownika	46
3.9	Logi i dane wyjściowe	46
3.10	Testy programu	50
3.11	Badanie wydajności programu	51
3.11.1	Użycie pamięci RAM	52
3.11.2	Użycie procesora	53
4	Badanie wpływu elementów algorytmu na wynik	57
4.1	Badanie operatora AEX	58
4.1.1	Rozmiar populacji	58
4.1.2	Prawdopodobieństwo mutacji	60
4.1.3	Ilość mutowanych osobników	60
4.1.4	Liczba podmian elementów genotypu	63
4.1.5	Liczba rodziców	64
4.1.6	Badanie operatora AEX dla większego magazynu	65
4.1.7	Podsumowanie badań operatora AEX	66
4.2	Badanie operatora ERX	66
4.2.1	Rozmiar populacji	67
4.2.2	Prawdopodobieństwo mutacji	67
4.2.3	Ilość mutowanych osobników	69
4.2.4	Liczba podmian elementów genotypu	70
4.2.5	Liczba rodziców	70
4.2.6	Badanie operatora ERX dla większego magazynu	72
4.2.7	Podsumowanie badań operatora ERX	72
4.3	Badanie operatora PMX	73
4.3.1	Rozmiar populacji	73
4.3.2	Prawdopodobieństwo mutacji	75
4.3.3	Ilość mutowanych osobników	76
4.3.4	Liczba podmian elementów genotypu	76
4.3.5	Badanie operatora PMX dla większego magazynu	78
4.3.6	Podsumowanie badań operatora PMX	79

4.4 Porównanie operatorów	79
5 Analiza wyników i wnioski	81
Spis algorytmów	83
Spis rysunków	83
Spis tabel	83
Bibliografia	86

Wykaz skrótów i oznaczeń

- AEX (Alternating Edges Crossover) - operator krzyżowania interpretujący genotyp jako ciąg skierowanych łuków.
- ERX (Edge Recombination Crossover) - operator krzyżowania interpretujący genotyp jako ciąg nieskierowanych krawędzi.
- PMX (Partially Mapped Crossover) - operator krzyżowania opierający się na dzieleniu genotypu na trzy części dwoma punktami cięcia, a następnie na tworzeniu mapy na podstawie której dokonywane jest krzyżowanie.
- MVVM (Model-View-ViewModel) - wzorzec projektowy stosowany w celu tworzenia aplikacji desktopowych gwarantujący oddzielność kodu backendowego od frontendowej implementacji za pomocą pośredniczącej warstwy ViewModel.

Wstęp

Bardzo ważnym elementem zarządzania magazynem jest optymalizacja czasu pracy pracowników, oraz minimalizacja kosztów obsługi. Najbardziej oczywistym podejściem do rozwiązania tego problemu jest zoptymalizowanie układu towaru w magazynie. Jeżeli faktycznie uda się poprawnie ułożyć towar pracownicy będą mogli obsługiwać magazyn efektywniej, co doprowadzi do spadku kosztów obsługi oraz wzrostu dochodów.

W tej pracy zaproponowane zostanie jedno z możliwych podejść do problemu optymalizacji układu towaru w magazynie z wykorzystaniem algorytmu genetycznego wielokryterialnego. Algorytmy genetyczne są często używanym rozwiązaniem w optymalizacji. Pozwalają szybko i efektywnie znaleźć rozwiązanie które dobrze spełnia zadane kryteria oceny, niezależnie od skomplikowania problemu.

Cel pracy

Celem pracy jest zaprojektowanie algorytmu genetycznego do optymalizacji układu towaru w magazynie. Algorytm ten będzie szukał rozwiązań na podstawie trzech kryteriów - popularności towarów, wagi towarów, oraz dystansów pomiędzy towarami powiązanymi ze sobą. Finalnie zostanie zbadana efektywność algorytmu w zależności od wybranych parametrów i konfiguracji.

Zakres pracy

1. Przeprowadzenie analizy literaturowej na temat algorytmów genetycznych oraz algorytmów genetycznych wielokryterialnych.
2. Definicja założeń projektowych.
3. Projekt algorytmu genetycznego do optymalizacji układu towaru w magazynie wraz z interfejsem użytkownika.
4. Testy wpływu parametrów i operatorów krzyżowania algorytmu na wynik.
5. Dyskusja otrzymanych wyników i wnioski.

1 Studium literaturowe z zakresu algorytmów genetycznych i algorytmów genetycznych wielokryterialnych

1.1 Podstawy algorytmów genetycznych

1.1.1 Optymalizacja

Optymalizacja to proces znajdowania rozwiązania najlepszego. Może być to zarówno minimum jak i maksimum. Najczęściej celem jest znalezienie globalnego optymalnego rozwiązania, czyli najlepszego rozwiązania w przestrzeni poszukiwań. Może być to problem trudny, w zależności od ograniczeń i wyglądu przestrzeni poszukiwań (np. bardzo dużej liczby lokalnych ekstremów). Dobrze zaprojektowany algorytm genetyczny potrafi rozwiązać problemy optymalizacji nawet z tymi ograniczeniami. W wielu wypadkach przestrzeń poszukiwań jest dyskretna, czyli problem należy do klasy optymalizacji kombinatorycznej. W drugim przypadku przestrzeń poszukiwań jest ciągła i może przyjmować wartości rzeczywiste. Aby można było dokonać optymalizacji potrzebna jest funkcja, zwana funkcją celu, funkcją kosztów, lub funkcją fitness. Niekoniecznie musi mieć ona zapis matematyczny. Optymalizacja to takie dobieranie parametrów funkcji aby uzyskać wartość najlepszą.

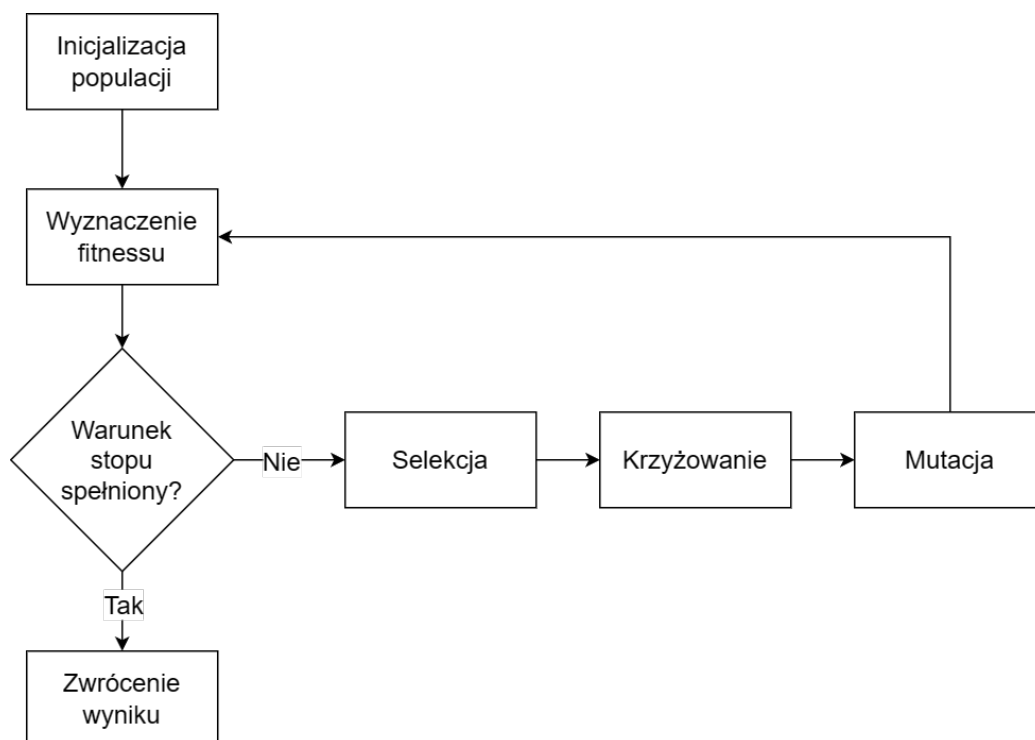
W praktyce często nie wyznacza się rozwiązania optymalnego, tylko rozwiązanie bardzo bliskie optymalnego, w zależności od przyjętej dokładności, lub złożoności problemu - w problemach bardzo trudnych można zadowolić się rozwiązaniem które mocno odstaje od optimum. W szczególności dla funkcji ciągłych znalezienie realnego punktu optymalnego może być ciężkie, ale wyznaczenie wartości bliskiej, nawet z bardzo dużą dokładnością, może być o wiele prostsze. [9] [11] [16]

1.1.2 Definicja i działanie algorytmu genetycznego

Algorytmy genetyczne to klasa modeli obliczeniowych wzorowanych na ewolucji. Próbuje one metaforycznie symulować naturalne procesy zachodzące w ewolucji, takie jak wymiana genów osobników, czy selekcja naturalna. W dużej mierze polegają na losowości, ale mają pewną sztywną strukturę, i tak jak w klasycznych algorytmach pewne operacje wykonywane są w pętli aż do znalezienia rozwiązania.

Działanie algorytmu genetycznego polega na utworzeniu populacji osobników (zbioru potencjalnych rozwiązań), a następnie rekombinacji genotypu tych osobników w celu utworzenia nowej, lepszej populacji. Proces przebiega iteracyjnie

aż zostanie spełniony warunek stopu. Najczęściej osobnika nazywa się chromosomem lub genotypem, co jest jednym z wielu nawiązań do ewolucji i genetyki. Konkretnie genotyp (lub chromosom) to zbiór cech osobnika. Działanie algorytmu genetycznego przedstawiono na rysunku 1.1. [8] [10] [13] [23]



Rysunek 1.1: Schemat działania algorytmu genetycznego.

Jak już wspomniano populacja to zbiór osobników, najczęściej stałej długości ciągów znaków. Każdy osobnik w populacji ma przypisaną wartość funkcji fitness. Populacja może być wstępnie wygenerowana losowo, ale żaden osobnik nie powinien łamać ograniczeń, jeżeli takie istnieją. Generowanie losowe jest akceptowalne, ale takie podejście grozi ryzykiem, że przestrzeń poszukiwań nie zostanie odpowiednio pokryta, przez co algorytm nigdy nie znajdzie ekstremum globalnego. Istnieją bardziej efektywne metody inicjalizacji wywodzące się ze statystyki, jednym z nich jest bezpośrednie wstrzyknięcie pożądanych cech do zainicjalizowanej populacji. [10] [16] [18]

Jak wspomniano wyżej, zakłada się, że każdy osobnik ma stałą długość. Bardzo często można się spotkać z kodowaniem osobnika w sposób binarny. Przykładowy osobnik może wyglądać następująco:

0 0 1 0 0 1 0 1 1 1 0 0 1 0 1 0 1

Taki osobnik może reprezentować bardzo wiele problemów. Przykładowo, gdy celem jest wybranie elementów zbioru, takie kodowanie może reprezentować które elementy zbioru zostały wybrane. 1 oznacza, że element został wybrany, a 0, że nie został wybrany.

Sposób kodowania osobnika powinien również uwzględniać ograniczenia, jeżeli takie w danym problemie istnieją. Przykład:

0 0 0 0

Zakładając, że jest to binarny zapis liczby dziesiętnej, to w praktyce oznacza, że dozwolony zakres wynosi od 0 (0000) do 15 (1111).

Kodowanie to bardzo ważny proces, bo od tego jak osobnik jest zakodowany zależy w jaki sposób algorytm wyznacza wynik, i jakie rozwiązania jest w stanie wyznaczyć. [9] [10] [23]

1.1.3 Operatory algorytmu genetycznego

Algorytm genetyczny może być skonstruowany na wiele różnych sposobów, ale pewne kroki muszą być wykonane zawsze. Te elementy algorytmu często nazywa się operatorami.

Operator fitness, zwany funkcją fitness, ocenia każdego z osobników pod kątem dostosowania, tzn. jakości rozwiązania. Jeżeli funkcja ma zapis matematyczny, to jest to problem prosty - wystarczy obliczyć jej wartość. Im mniejsza lub większa wartość funkcji (w zależności od tego, czy to problem minimalizacji czy maksymalizacji) tym lepsze jest to rozwiązanie. Wartość fitness jest jedyną informacją którą otrzymuje algorytm w celu znalezienia rozwiązania. Wygląd funkcji fitness zależy od procesu modelowania algorytmu. Bardzo często można do niej wprowadzić takie elementy jak ograniczenia, np. za pomocą funkcji kary. Ilość wywołań funkcji fitness w każdej iteracji algorytmu powinna być jak najmniejsza, w szczególności gdy wiąże się z tym duży narzut czasowy. [11] [17]

Operator selekcji ma za zadanie dobierać osobniki do krzyżowania. Najczęściej zakłada się, że wybierane są dwa osobniki, ale istnieją operatory które mogą zwrócić więcej osobników. Niezależnie od tego, jakie jest konkretne podejście, idea jest ta sama - z większym prawdopodobieństwem wybierane są osobniki lepiej dostosowane, tzn. te o lepszej wartości funkcji fitness. To z kolei prowadzi do polepszenia wartości funkcji fitness w następnych pokoleniach. Aczkolwiek nie można wybierać tylko osobników najlepszych, gdyż szybko może dojść do wyjąłowienia populacji i utraty różnorodności. Przykładowe operatory selekcji to selekcja turniejowa, selekcja ruletki, selekcja losowa, selekcja rangi, selekcja Boltzmanna. [9] [18]

Operator krzyżowania służy tworzeniu nowej populacji. Gdy wybrano już osobniki za pomocą selekcji (jak wspomniano wyżej - najczęściej dwa, ale może być

ich więcej), następuje wymiana materiału genetycznego, tj. genotypu. Najprostszym podejściem jest operator one-point crossover, który dzieli dwa osobniki na dwie części, i łączy te części w nowe dwa (lub jednego) osobnika. Ogólnie operatory krzyżowania najczęściej zwracają jednego lub dwa osobniki, ale istnieją operatory które potrafią wygenerować ich więcej. Proces jest powtarzany tak długo, aż utworzona zostanie nowa populacja. Przed każdym uruchomieniem operatora krzyżowania powinno się dostarczać nowe osobniki za pomocą operatora selekcji. Logika postępowania operatora krzyżowania jest bardzo prosta - jeżeli wymiesza się elementy dobrze dostosowanych osobników, to nowo utworzony osobnik ma szansę być jeszcze lepszy niż jego rodzice. Nie istnieje skuteczna metoda na wybór odpowiedniego operatora krzyżowania. Na jego działanie wpływa bardzo duża liczba zmiennych, i najlepiej jest wybrać go w sposób eksperymentalny. Wyjątkiem jest wybór kategorii operatora - niektóre operatory działają dla genotypów które są permutacją, inne zakładają, że w genotypie mogą występować duplikaty elementów (np. ciąg zer i jedynek). [11] [12]

Operator mutacji wymusza odpowiednią różnorodność w populacji, i chroni przed zjawiskiem przedwczesnej konwergencji. Jest to zjawisko w którym algorytm niepoprawnie przeszukuje przestrzeń poszukiwań, i zaczyna zbiegać się do ekstremum lokalnego. Operator mutacji dokona w losowy sposób rozrzutu osobników w przestrzeni poszukiwań, co jak wspomniano wyżej chroni przed utratą różnorodności. Najprostszym podejściem do mutacji jest losowe podmienianie elementów genotypu. W przypadku kodowania binarnego można losowo 0 zmieniać na 1, i 1 na 0. W przypadku gdy osobniki to permutacje jakiegoś zbioru, można losowo zamieniać elementy miejscami. Istnieje wiele innych podejść do operatora mutacji, przykładowo można zaprojektować matematyczną funkcję mutacji, lub mutować osobniki za pomocą maski. Należy również dodać, że stosowanie operatora mutacji nie jest obligatoryjne, ale warto go wprowadzić gdy występują problemy ze zbieganiem rozwiązania do ekstremum lokalnego. [13] [16]

1.1.4 Warunek stopu

W teorii algorytm genetyczny jest w stanie działać w nieskończoność. W praktyce musi w pewnym momencie zwrócić rozwiązanie. Warunek stopu (zwany niekiedy terminacją) definiuje kiedy algorytm genetyczny powinien zakończyć swoje działanie i zwrócić wynik, co oznacza, że znalazł rozwiązanie o zadowalającym przybliżeniu. Istnieje kilka typowych rozwiązań.

Najprostszym podejściem jest uruchomienie algorytmu z góry określoną liczbą iteracji. Możliwe jest również nastawienie konkretnego czasu działania (np. 30 minut).

Innym podejściem jest takie zaprojektowanie algorytmu, aby zatrzymał się gdy nie znajduje już rozwiązania lepszego, lub nowe rozwiązania są niewiele

lepsze o poprzedniego (w zależności od przyjętej dokładności). Podobnie można wyznaczać różnorodność populacji, i zatrzymać algorytm gdy różnorodność zacznie maleć, aż spadnie poniżej pewnego progu.

Gdy algorytm znalazł już rozwiązanie, wszystkie osobniki w populacji będą bardzo podobne, albo nawet identyczne, gdy nie uwzględni się mutacji. Dlatego podejście ze stałą liczbą iteracji nie jest zbyt praktyczne, gdyż algorytm niepotrzebnie dokonuje obliczeń pomimo znalezionego wyniku. O wiele lepiej jest w jakikolwiek sposób próbować wyznaczyć czy algorytm znalazł już rozwiązanie, i wtedy dokonać terminacji. [9] [16]

1.2 Podstawy algorytmów genetycznych wielokryterialnych

1.2.1 Optymalizacja wielokryterialna

Celem optymalizacji wielokryterialnej jest znalezienie optymalnego balansu pomiędzy kryteriami, które często przeczą sobie nawzajem (tzn. polepszanie jednego prowadzi do pogarszania drugiego). Metody rozwiązywania takich problemów będą eksplorować przestrzeń poszukiwań szukając kompromisu. W takich problemach optimum globalne nie istnieje. Istnieje za to zbiór rozwiązań optymalnych. Każde z nich jest optymalne w sensie Pareto, tzn. że nie istnieje rozwiązanie które jest lepsze ze względu na co najmniej jedno kryterium, i jednocześnie nie gorsze ze względu na inne kryteria. Można spotkać się z nazwą rozwiązania kompromisowe. Mimo, że każde z nich jest już optymalne w sensie Pareto, to trzeba zdecydować się na pewien kompromis, gdyż różne rozwiązania będą w inny sposób spełniały różne kryteria. Istnienie wielu rozwiązań optymalnych jest kluczową różnicą pomiędzy optymalizacją jednokryterialną i wielokryterialną, która prowadzi do pewnych zmian w procesie szukania rozwiązania. Wybór najlepszego rozwiązania ze zbioru rozwiązań optymalnych w sensie Pareto również może być trudnym problemem. Osoba podejmująca decyzję musi wybrać które rozwiązanie jest najbardziej zadowalające w danym kontekście. W przypadku optymalizacji kombinatorycznej ilość rozwiązań jest skończona, ale może być bardzo duża. W przypadku optymalizacji ciągłej takich rozwiązań jest nieskończenie wiele. [7] [19]

1.2.2 Działanie algorytmu genetycznego wielokryterialnego

Algorytm genetyczny wielokryterialny to w dalszym ciągu algorytm genetyczny, i teoria opisana w poprzednim rozdziale dalej obowiązuje. Główną zmianą jest podejście do funkcji fitness, co prowadzi do kilku kolejnych zmian. Istnieje kilka podejść do implementacji algorytmu genetycznego wielokryterialnego, i nie wszystkie istniejące metody zostaną tutaj omówione.

Algorytm genetyczny zakłada tylko jedno kryterium oceny - funkcję fitness.

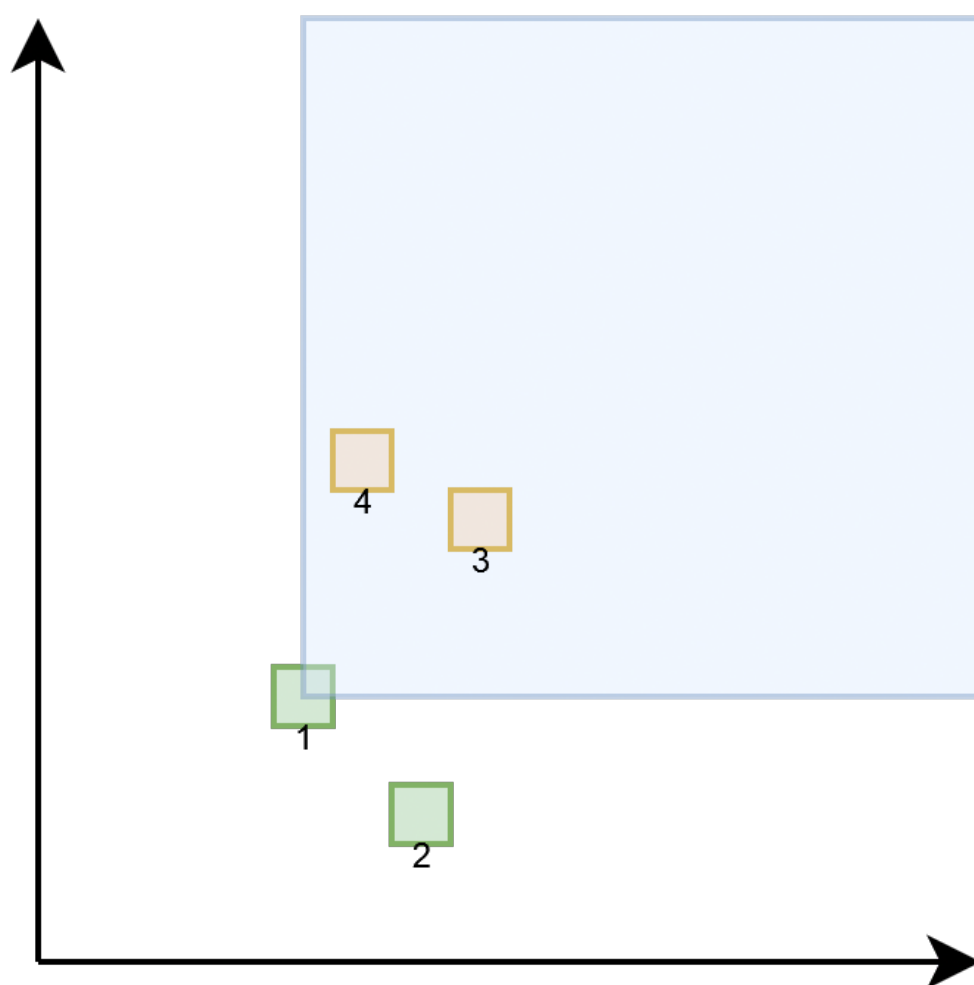
W problemie z wieloma kryteriami takich funkcji będzie wiele, każda kolejna funkcja fitness reprezentuje inne kryterium. Klasyczny operator selekcji w takim przypadku nie jest w stanie określić które osobniki są lepsze, a które gorsze.

Częstym rozwiązaniem problemu wielu kryteriów jest sprowadzenie takiego problemu do optymalizacji jednokryterialnej. Jednym z możliwych podejść jest użycie sumy ważonej kryteriów, gdzie waga zależy od ważności kryterium, w celu utworzenia pojedynczej funkcji celu. Takie podejście nie jest błędne, a niekiedy może być nawet efektywne, ale trzeba go używać z rozwagą. Znalezione rozwiązanie optymalne w dużej mierze zależy od tego, jakie wagi dobrano i w jaki sposób sprowadzono wiele kryteriów do jednego kryterium. Używanie pełnoprawnej optymalizacji wielokryterialnej można uznać za bezpieczniejsze. Nie występują tutaj problemy ze sprowadzaniem wielu kryteriów do jednego kryterium. Poprawne wykonanie tego zadania wymaga doświadczenia, i często jest subiektywne, zależy od tego kto będzie go wykonywał. A w przypadku zmiany wymagań proces należy wykonać ponownie. Kolejną wadą podejścia sumy ważonej jest fakt, że przy sprowadzaniu wielu kryteriów do jednego kryterium istnieje prawdopodobieństwo utraty części przestrzeni poszukiwań. [4] [7] [19]

W przypadku istnienia wielu kryteriów będą tworzyć się fronty Pareto. Dla dwóch kryteriów są to krzywe, dla trzech - powierzchnie. Ogólnie mówiąc są n -wymiarowe, gdzie n to liczba kryteriów. Wszystkie rozwiązania na jednym froncie są optymalne w sensie Pareto. Używa się również określenia zdominowania. Wszystkie rozwiązania na jednym froncie są niezdominowane, czyli jak wspomniano wyżej - optymalne w sensie Pareto. Jeżeli istnieje rozwiązanie które jest lepsze pod względem co najmniej jednego kryterium, i jednocześnie nie gorsze pod względem żadnego innego to znaczy, że dominuje ono nad innymi rozwiązaniami, które mają gorsze wartości kryteriów. Jeżeli jedno rozwiązanie dominuje nad drugim, nie mogą znajdować się na tym samym froncie Pareto.

Wizualizacja dominacji Pareto dla problemu minimalizacji przedstawiono na rysunku 1.2. Rozwiązania zielone są na pierwszej krzywej, a pomarańczowe - na drugiej. Rozwiązania 3 i 4 nie mogą być na tej samej krzywej co rozwiązanie 1, bo są przez nie zdominowane. Rozwiązanie 2 nie dominuje nad 1, ani 1 nad 2, dlatego mogą być na tej samej krzywej. Rozwiązanie 2 nie dominuje nad rozwiązaniem 4, ale jak wspomniano wyżej i tak nie może być na tej samej krzywej, bo zdominowało go rozwiązanie 1.

Wprowadza się tutaj nową wartość zwaną rangą, której jedynym celem jest definiowanie na którym froncie Pareto znajduje się osobnik. Wszystkie osobniki na jednym froncie mają tę samą rangę. Jeżeli jeden osobnik ma niższą rangę od drugiego, tzn. że jest lepszym rozwiązaniem, i znajduje się na innym (w przypadku minimalizacji - niższym) froncie Pareto. Które fronty dostaną którą rangę zależy już od tego, czy jest to problem maksymalizacji czy minimalizacji, ale zawsze będą to wartości rosnące lub malejące.



Rysunek 1.2: Dominacja Pareto - wizualizacja.

Jak można się domyślić, samo wprowadzenie rangi nie jest jeszcze wystarczające, gdyż nie da się określić które osobniki są lepsze a które gorsze w konkretnym froncie. Wprowadza się jeszcze drugą wartość - odległość zagęszczenia. Istnieje konkretny algorytm jej wyznaczania:

- Załóż, że $I=|F|$, gdzie I to rozwiązania we froncie F ,
- Dla każdego i w I nadaj wartość $d_i = 0$,
- Dla każdej funkcji celu posortuj zbiór rosnąco według fitnessu, lub znajdź posortowany wektor indeksów: $I^m = \text{sort}(f_m, >)$,
- Dla każdej funkcji celu nadaj dużą odległość dla rozwiązań granicznych lub ∞ , a dla reszty rozwiązań $j = 2$ do $(l-1)$ nadaj: $d_{I_j^m} = d_{I_j^m} + \frac{f_m^{(I_{j+1}^m)} - f_m^{(I_{j-1}^m)}}{f_m^{\max} - f_m^{\min}}$

Algorytm może być trudny do zrozumienia na pierwszy rzut oka, dlatego wizualizację przedstawiono na rysunku 1.3

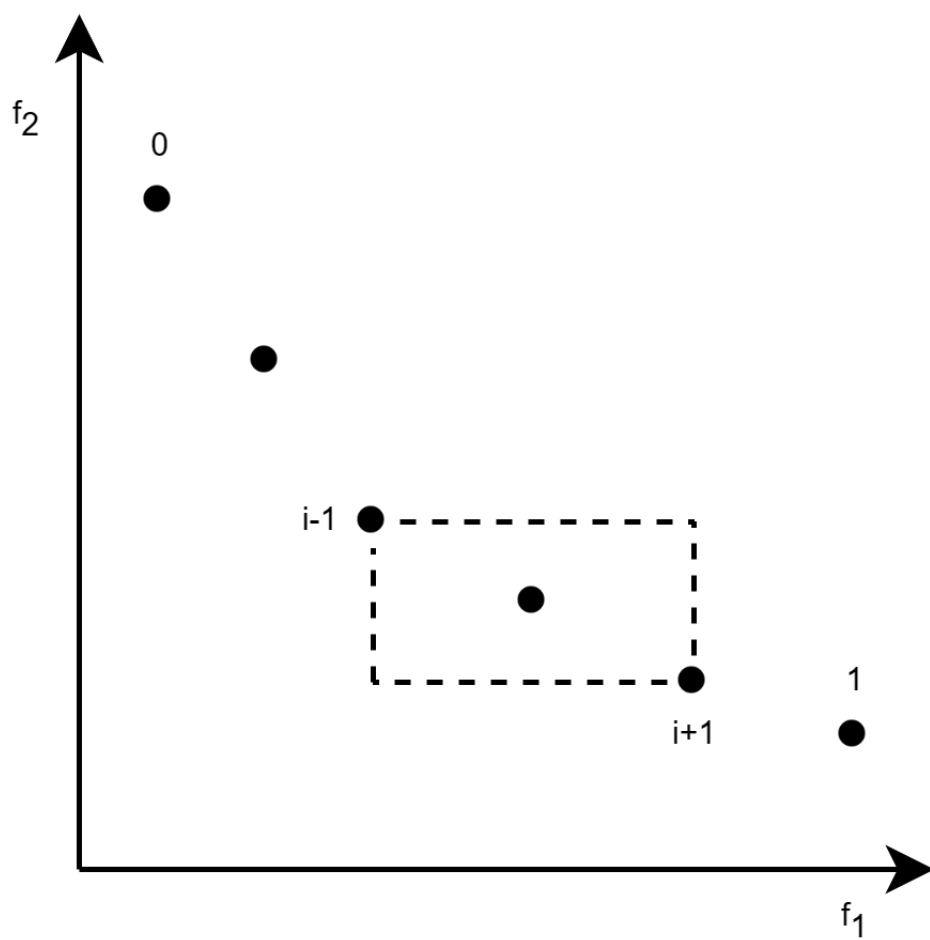
Grupy osobników otrzymają niską wartość, osobniki odosobnione dostaną wyższą wartość, a osobniki na granicach frontu - dodatnią nieskończoność. Im wyższa wartość, tym większe prawdopodobieństwo selekcji. Wynika to z tego, że osobniki zgrupowane naturalnie mają wyższe prawdopodobieństwo selekcji, a osobniki odosobnione i graniczne mają potencjalnie dobre cechy, dlatego warto nadać im wyższy priorytet. [1] [7] [19] [20]

Ranga i odległość zagęszczenia to wystarczające informacje dla operatora selekcji. Aczkolwiek operator selekcji zakłada istnienie jednej wartości, dlatego potrzebne są pewne poprawki. Efektywnym podejściem jest stosowanie operatora selekcji turniejowej Crowded Tournament Selection, która uwzględnia rangę i odległość zagęszczenia. Podejście jest bardzo podobne do klasycznego operatora selekcji turniejowej, ale wygrywa osobnik z niższą rangą, a gdy rangi są takie same - z wyższą odległością zagęszczenia.

Jak wspomniano wyżej, w algorytmie wielokryterialnym istnieje wiele optymalnych rozwiązań. Po utworzeniu najlepszego frontu i zwróceniu wyniku, zwrócony zostanie zbiór rozwiązań, i wszystkie z nich są optymalne w sensie Pareto. Istnieje kilka podejść wyboru które rozwiązanie jest rozwiązaniem pożądanym. Osoba decyzyjna manualnie może zdecydować, które rozwiązanie jej odpowiada, analizując wyniki i wartości kryteriów. Można próbować również zastosować metody automatycznego wyboru, przykładowo sumę ważoną, lub sumę znormalizowanych wartości kryteriów, gdy kryteria są równie ważne. [6] [7] [21]

1.3 Analiza istniejących rozwiązań

Problem optymalizacji układu towaru w magazynie z wykorzystaniem algorytmu genetycznego nie jest nowy. Istnieje już wiele sugerowanych rozwiązań



Rysunek 1.3: Odległość zagęszczenia - wizualizacja.

tego problemu. Zostaną one przeanalizowane w tym podrozdziale.

Avdekins i in. [3] proponują następujące rozwiązanie - magazyn reprezentowany jest jednowymiarowo, i każdy regał może pomieścić jeden unikatowy towar. Osobnik kodowany jest jako permutacja układu towaru w magazynie, to znaczy ciąg liczb całkowitych reprezentujących towary. Fitness obliczany jest na podstawie listy towarów, a pod uwagę z każdej listy brany jest towar najdalej ustawiony od wejścia. Do selekcji wybierane jest tylko 30% najlepszych osobników. Zastosowano operator krzyżowania OX (Order Crossover), oraz mutację 5% nowych osobników.

Rozwiązanie proponowane przez Qina i in. [15] uwzględnia wiele kryteriów. Magazyn reprezentowany jest przez dwuwymiarową macierz, a regały są wielopoziomowe. Co za tym idzie osobnik nie jest już jednowymiarowym ciągiem liczb, a dwuwymiarową tablicą. Algorytm uwzględnia wagę towarów, dystanse pomiędzy towarami, oraz wartość efektywności zarządzania magazynem, która z kolei uwzględnia częstość występowania towarów i czas pracy osoby wykonującej zadanie w magazynie. Użyto specjalnie dostosowanych operatorów selekcji i krzyżowania, oraz mutację polegającą na losowej podmianie elementów genotypu.

Wang i in. [22] rozwiązują problem uwzględniając wiele kryteriów, ale minimalizowane są do jednej funkcji. Uwzględniana jest rotacja towaru (częstość wynoszenia i wnoszenia), stabilność regału, oraz korelacja towarów (towary powiązane ze sobą). Osobniki kodowane są jednowymiarowo, ale każda komórka tabeli zawiera trójwymiarowe koordynaty towaru. Zaimplementowano selekcję ruletki i krzyżowanie za pomocą przełączania jednopunktowego. Prawdopodobieństwa używane w mutacji i krzyżowaniu są dynamiczne.

Cheng i in. [5] opracowują problem trochę inny, mianowicie optymalizowanie procesu odkładania towarów na regały w magazynie. Proponują optymalizację na podstawie następujących trzech kryteriów - maksymalizacja korelacji pomiędzy przedmiotem odbieranym a przedmiotami w zadanej ramie magazynowej (regale), maksymalizacja wskaźnika częstotliwość-odległość opartego na koncepcji CPOI, oraz minimalizowanie odległości ruchomej pomiędzy przydzielonymi ramami magazynowymi. Wynikają one z reguł przydzielania miejsca w magazynie - przedmioty o wysokim wskaźniku rotacji są ustawienie bliżej wyjścia, przedmioty skorelowane są ustawiane sąsiadująco, oraz towary uzupełniające są ustawiane również sąsiadująco (aby zastąpiły inne towary w przypadku braku towaru na stanie). Osobnik kodowany jest jako ciąg odwiedzanych regałów. Użyto selekcji ruletki, oraz selekcji turniejowej. Zastosowano operator krzyżowania zwany one-cut-ratio, który jest autorską wersją one-point crossover dla genotypów o różnej długości. Mutacja polega na losowej zamianie komórki genotypu w zależności od ustalonego prawdopodobieństwa. Proces nie jest stuprocentowo losowy, aby nie pogarszać wyniku.

1.4 Analiza porównawcza przedstawionych rozwiązań

Porównanie istniejących algorytmów przedstawiono w tabeli 1.1 i 1.2. Jak widać mimo, że algorytm genetyczny ma pewną sztywną strukturę, to implementacja konkretnych elementów mocno się różni. Osobnik najczęściej reprezentowany jest jako jednowymiarowa permutacja pozycji towarów. Metoda pomiaru jakości układu towaru jest różna, a waga nie jest uwzględniana zawsze. Metoda selekcji jak i krzyżowania jest bardzo różna, i najprawdopodobniej dobór wynika z doświadczenia twórcy, lub jest eksperymentem. Mutacja zawsze polega na losowej podmianie elementów genotypu, ale konkretna implementacja i w szczególności prawdopodobieństwo różni się. Można również zauważyć, że nawet dla problemu optymalizacji procesu odkładania towarów na regały podejście jest stosunkowo podobne, i zastosowane kryteria oceny nie odbiegają od problemów skupionych na optymalizacji układu towaru w magazynie.

Autor algorytmu	Regały	Osobnik	Kryteria
Avdekins	Jednopoziomowe	Jednowymiarowy	Dystans od wejścia
Qin	Wielopoziomowe	Wielowymiarowy	Waga, dystans pomiędzy towarami, efektywność operowania magazynem
Wang	Wielopoziomowe	Mieszany	Rotacja towaru, stabilność regału, korelacja towarów
Cheng	Jednopoziomowe	Jednowymiarowy	Rotacja towaru, korelacja towarów, uwzględnienie towarów uzupełniających

Tabela 1.1: Tabela porównania algorytmów - model.

Autor algorytmu	Selekcja	Krzyżowanie	Mutacja
Avdekins	30% najlepszych osobników	OX	5% nowych osobników
Qin	Gamble selection	Mapowanie	Podmiana dwóch losowych elementów z danym prawdopodobieństwem
Wang	Metoda ruletki	Przełączanie jednopunktowe	Dynamiczna
Cheng	Metoda ruletki, selekcja turniejowa	One-cut-ratio	Podmiana losowego elementu genotypu

Tabela 1.2: Tabela porównania algorytmów - operatory.

2 Definicja założeń do realizacji projektu

W tym rozdziale zostaną zdefiniowane założenia projektowe, tzn. co i jak ma zostać wykonane.

Wykonany ma zostać algorytm genetyczny który optymalizuje układ towaru w magazynie uwzględniający trzy elementy:

- Popularność towaru,
- Wagę towaru,
- Odległość towaru od innych towarów, jeżeli towary te są ze sobą związane (na przykład często kupowane razem).

Szczegóły zdefiniowane zostaną w następnych podrozdziałach.

2.1 Platforma programistyczna i język programowania

Zdecydowano się na użycie platformy .NET i języka C#. Jest to wysokopoziomowy język pozwalający na szybkie i efektywne wytwarzanie wysokiej jakości oprogramowania. Skompilowany kod nie odstaje zbyt mocno szybkością wykonywania od takich języków jak C czy C++, gdzie implementacja takiego algorytmu byłaby procesem czasochłonnym i żmudnym. Dodatkowo .NET posiada zestaw narzędzi pozwalających na szybkie i łatwe implementowanie interfejsów użytkownika. Aczkolwiek w przypadku gdzie czas działania jest absolutnym priorytetem, język C lub C++ byłby lepszym wyborem. Należy jednak pamiętać o wysokim poziomie trudności tych języków, oraz potrzebie własnej implementacji wielu bazowych elementów, które są już wbudowane w inne języki wysokopoziomowe.

2.2 Format danych wejściowych

Aby algorytm genetyczny był w stanie wykonać jakiegokolwiek obliczenia, potrzebuje danych wejściowych. Potrzebne są dwa rodzaje informacji - dane na temat towarów, oraz dane na temat struktury magazynu. Wszystkie rodzaje danych wejściowych mogą być zapisywane w formacie pliku tekstowego. Proponowane są następujące formaty danych.

2.2.1 Lista towarów

Lista towarów wraz z identyfikatorem, popularnością, oraz wagą. Format ogólny:

ID , Popularity , Weight

Przykład:

1,100,1
2,80.5,3.2
3,60,5
4,40,7
5,20,10
6,5,20

ID to to identyfikator towaru i jest liczbą całkowitą. Powinien zaczynać się od 1. *Popularity* to liczba zmiennoprzecinkowa podwójnej precyzji (np. 2.5, z czego bardzo ważne jest używanie kropki a nie przecinka). Popularność to pewna abstrakcyjna wartość i może oznaczać wiele rzeczy w zależności od faktycznych danych wejściowych. Może to być realna ilość zamówień konkretnego towaru, współczynnik obrotu towarem, znormalizowana ilość zamówień (np. od 1 do 10), i tak dalej.

Weight to liczba zmiennoprzecinkowa podwójnej precyzji (ponownie powinna być używana kropka zamiast przecinka), i ponownie jest to pewna abstrakcyjna wartość. Może oznaczać realną wagę towaru (w spójnej jednostce wybranej przez użytkownika), lub wagę znormalizowaną (np. od 1 do 10).

2.2.2 Lista towarów powiązanych

Lista towarów powiązanych zawiera w każdej linii osobną listę wraz z ważnością związku. Waga to pierwszy element w linii, każdy kolejny to identyfikator towaru (musi znajdować się na liście towarów). Format ogólny:

Weight , Orders

Przykład:

250,1,5,15,25
125,4,12,18

Weight nie jest tą samą wagą co waga z listy towarów, jest to symboliczna nazwa ważności danej listy zamówień. Jest to liczba całkowita. W praktyce może to być realna ilość zamówień takiego zestawu towarów, lub abstrakcyjna znormalizowana wartość (przykładowo 10 dla zestawu bardzo popularnego, a 1 dla zestawu mało popularnego). *Orders* to każdy następny element po pierwszym przecinku, i jako identyfikator towaru z listy musi być liczbą całkowitą.

2.2.3 Struktura magazynu

Jeden plik zawierający pełną strukturę magazynu. Format ogólny:

Rack_ID : Rack_Size — Connections : Weights

Przykład:

1 : 3 — 0,2 : 1.5, 1
2 : 3 — 1,3 : 1,1
3 : 3 — 2 : 1

Rack_ID to liczba całkowita reprezentująca identyfikator regału. Powinna zaczynać się od 1.

Rack_Size to ilość półek (poziomów) na regale, musi to być liczba całkowita.

Connections to dozwolone przejścia do innych regałów z danego regału. Jest to liczba całkowita.

Weights to koszty przejścia dla każdego odpowiadającego dozwolonego przejścia. Jest to liczba zmiennoprzecinkowa podwójnej precyzji. Może to być realny dystans np. w metrach, lub pewna symboliczna wartość.

Popularną metodą zapisu magazynu jest również macierz, dlatego proponowana jest jeszcze alternatywna metoda zapisu danych, przedstawiona w dwóch następnych podrozdziałach.

2.2.4 Struktura magazynu - macierz

Macierz zawierająca informację na temat struktury magazynu, tj. możliwe przejścia, i koszty przejść. Przykład:

0 1.5 0 0
1.5 0 1 0
0 1 0 1
0 0 1 0

Macierz działa analogicznie do struktury magazynu zdefiniowanego w poprzednim podrozdziale. Każdy wiersz i każda kolumna to odpowiedni regał. Każda komórka macierzy posiada wagę przejścia (liczbę zmiennoprzecinkową podwójnej precyzji) lub 0 gdy przejście nie jest dozwolone.

2.2.5 Struktura magazynu - regały

Plik zawierający informację na temat regałów w magazynie, tj. ich identyfikatory oraz ilość pięter. Format ogólny:

Rack_ID : *Rack_Size*

Przykład:

1 : 3
2 : 3
3 : 4

Analogicznie jak w przypadku jednego pliku opisującego strukturę magazynu *Rack_ID* to liczba całkowita będąca identyfikatorem regału. *Rack_Size* to ilość półek na regale.

Dozwolony jest również zapis skrócony, który pozwala zdefiniować wiele regałów w jednej linii:

`Rack_ID_First — Rack_ID_Last : RackSize`

Przykład:

1 — 3 : 3
4 — 6 : 4

Ten przykład oznacza, że regały od 1 do 3 mają 3 półki, a regały od 4 do 6 mają 4 półki.

2.3 Kodowanie osobnika

Osobnik zakodowany jest jako ciąg liczb całkowitych reprezentujących identyfikatory towarów. Każdy osobnik jest permutacją listy towarów, i reprezentuje inny układ towaru w magazynie. Kodowanie przedstawiono na rysunku 2.1. Diagram przedstawiony jest dwuwymiarowo. Regały oznaczone są literą *R*, a piętra literą *L*. Wizualizuje to również w jaki sposób można rozpoznać pozycję towaru w magazynie na podstawie pozycji w genotypie. Na przykładzie towar na pozycji 1 w genotypie będzie na pozycji *R1 L1* w magazynie. Wystarczy utworzyć strukturę danych która pamięta strukturę magazynu.

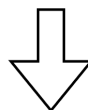
Kodowanie przebiega po każdym regale, i po każdej półce regału. Patrząc na diagram - kodowanie zaczyna się od regału pierwszego *R1*, do genotypu wpisywane są po kolei wszystkie piętra regału od dołu do góry - 1, 2, oraz 3. Regał ten nie ma więcej pięter, więc następuje przeskok do regału drugiego *R2* i znowu wpisywane są kolejne piętra.

Alternatywną opcją jest kodowanie najpierw po każdym poziomie, i dopiero po zakodowaniu poziomu pierwszego każdego regału następuje przejście na poziom drugi, etc. Taki sposób kodowania może prowadzić do problemów podczas krzyżowania jak i mutacji, gdyż ciężiej będzie zamieniać produkty między poziomami regałów. W proponowanym kodowaniu nie będzie to problemem, gdyż niewielka zmiana genotypu może prowadzić do zmian pozycji zarówno na regale, jak i na piętrze regału.

2.4 Operatory algorytmu genetycznego

Poniżej zdefiniowane zostaną założenia operatorów algorytmu genetycznego. Ich konkretna implementacja zostanie opisana w następnym rozdziale.

	L3	L2	L1
R1	3	2	1
R2	6	5	4
R3	9	8	7
R4	12	11	10
R5	15	14	13



1	2	3	4	...	12	13	14	15
---	---	---	---	-----	----	----	----	----

Rysunek 2.1: Kodowanie osobnika.

2.4.1 Fitness i kryteria oceny

W operatorze funkcji fitness należy uwzględnić trzy elementy - popularność towaru, wagę towaru, oraz odległość towaru od innych towarów, jeżeli towary są ze sobą powiązane. Nietrudno domyślić się, że wartości te mogą mieć zupełnie inne rzędy wielkości, oraz są całkowicie niezależne. To oznacza, że nie da się zdefiniować poprawnej funkcji fitness. Dlatego konieczne będą trzy niezależne kryteria oceny.

Kryterium pierwsze, czyli popularność towaru, musi mieć mniejszą wartość jeżeli popularny towar jest bliżej wyjścia z magazynu. Można to zdefiniować następującą funkcją:

$$\sum_{i=1}^n (p_i \cdot d_i + p_i \cdot h_i)$$

Gdzie:

- i to element genotypu,
- n to długość genotypu,
- p_i to wartość popularności towaru na pozycji i ,
- d_i to dystans od wejścia na pozycji i ,
- h_i to wysokość na regale na pozycji i .

Kryterium drugie, czyli waga towaru, analogicznie musi mieć mniejszą wartość jeżeli ciężki towar jest blisko wyjścia z magazynu. Można to opisać bardzo podobną funkcją:

$$\sum_{i=1}^n (w_i \cdot d_i + w_i \cdot h_i)$$

Gdzie:

- i to element genotypu,
- n to długość genotypu,
- w_i to wartość wagi towaru na pozycji i ,
- d_i to dystans od wejścia na pozycji i ,
- h_i to wysokość na regale na pozycji i .

Ostatnie kryterium to odległość towarów od siebie jeżeli występują na tych samych listach definiujących ich powiązanie. Im bliżej są siebie, tym mniejsza wartość kryteriów. Zdefiniować to można następującym wzorem:

$$\sum_{i=1}^n \sum_{j=1}^m \sum_{k=1}^l (dist(t_i, t_{jk}))$$

Gdzie:

- i to element genotypu,
- n to długość genotypu,
- j to numer listy towarów,
- m to ilość list towarów powiązanych,
- k to element listy towarów,
- l to długość listy towarów,
- t to konkretny towar na liście,
- $dist(a, b)$ to odległość produktu a od b w magazynie.

Kryteria oceny zostały zdefiniowane, i będą rozpatrywane niezależnie, zgodnie z założeniami algorytmu genetycznego wielokryterialnego. Jak już ustalono w rozdziale pierwszym, podejście sprowadzenia problemu do jednej funkcji celu niesie ze sobą wiele potencjalnych problemów ze znalezieniem dobrego wyniku, dlatego ta opcja nie zostanie użyta. Zostanie dodane również wyznaczanie rangi i odległości zagęszczenia, aby algorytm genetyczny wielokryterialny był w stanie funkcjonować. Na tej podstawie algorytm może dokonać selekcji.

2.4.2 Operator selekcji i mutacji

Biorąc pod uwagę, że zastosowany jest algorytm wielokryterialny, należy zastosować specjalny operator selekcji który weźmie pod uwagę rangę oraz odległość zagęszczenia. Zaimplementowany zostanie operator selekcji turniejowej, zwany Crowded Tournament Selection. Działa on tak samo jak zwykły operator selekcji turniejowej, ale uwzględnia dodatkowo rangę i odległość zagęszczenia. Do turnieju losowane są dwa osobniki. Turniej wygrywają te osobniki, które mają mniejszą rangę i większą odległość zagęszczenia.

Jeśli chodzi o operator mutacji, to zaimplementowany zostanie prosty operator który losowo podmienia elementy genotypu osobników.

2.4.3 Operatory krzyżowania

W pracy tej zostanie wykonany przegląd operatorów krzyżowania w celu określenia który z nich daje najlepsze rezultaty. Proponowane operatory to AEX, ERX, oraz PMX. Ich cechą wspólną jest ich przeznaczenie do problemów gdzie osobnik to permutacja pewnej sekwencji.

AEX, tj. Alternating Edges Crossover interpretuje genotyp jako ciąg skierowanych łuków, i po kolei pobiera łuki od rodziców, przełączając się pomiędzy nimi. Jeżeli następny łuk zamknie cykl, należy pobrać losowy element który nie istnieje jeszcze w potomku. AEX nie ma ograniczeń co do liczby rodziców, ale może stworzyć tylko jednego potomka.

Przykład:

$$p_1 = (2\ 3\ 1\ 4\ 5\ 8\ 7\ 6),$$

$$p_2 = (5\ 1\ 2\ 8\ 3\ 4\ 6\ 7).$$

Najpierw pobierany jest pierwszy łuk z pierwszego rodzica

$$c = (2\ 3\ *\ *\ *\ *\ *).$$

Następnie kolejny łuk z następnego rodzica

$$c = (2\ 3\ 4\ *\ *\ *).$$

Operator kontynuuje przełączając się pomiędzy rodzicami

$$c = (2\ 3\ 4\ 5\ 1\ *).$$

Następny powinien być łuk od 1 do 4, ale zamknąłby on cykl. Losowo wybierany jest element 6

$$c = (2\ 3\ 4\ 5\ 1\ 6).$$

Proces analogicznie prowadzony jest do końca

$$c = (2\ 3\ 4\ 5\ 1\ 6\ 7\ 8).$$

ERX, tj. Edge Recombination Crossover interpretuje genotyp jako ciąg nieskierowanych krawędzi. Operator ma za zadanie zagwarantować, że potomek posiada jak najwięcej krawędzi rodziców, w szczególności tych wspólnych dla obu rodziców. Jest dedykowany głównie dla Travelling Salesman Problem i pochodnych, dlatego jego implementacja będzie testem, czy dla tego problemu również będzie efektywny. Operator działa dla wielu rodziców, ale może zwrócić tylko jednego potomka.

Przykład:

$$p_1 = (2\ 3\ 1\ 4\ 5\ 8\ 7\ 6),$$

$$p_2 = (5 \ 1 \ 2 \ 8 \ 3 \ 4 \ 6 \ 7).$$

Najpierw pobierany jest pierwszy element pierwszego rodzica, w tym wypadku 2. Sąsiadami dwójki są 3, 6, 1, 8, i każdy z tych elementów ma 4, 3, 4, 4 sąsiadów. Najmniejszą liczbę ma element 6, dlatego jest następnym wybranym elementem

$$c = (2 \ 6 \ * \ * \ * \ * \ * \ *).$$

Sąsiadami 6 są 2, 7, 4, i mają oni 4, 3, 4 sąsiadów. 2 jest już w genotypie, a najmniej sąsiadów ma element 7, dlatego zostaje wybrany

$$c = (2 \ 6 \ 7 \ * \ * \ * \ * \ *).$$

Proces jest analogicznie kontynuowany do końca

$$c = (2 \ 6 \ 7 \ 5 \ 1 \ 4 \ 3 \ 8).$$

PMX, tj. Partially Mapped Crossover dzieli genotyp na trzy części za pomocą dwóch losowych punktów. Część pomiędzy punktami cięcia z pierwszego rodzica jest przeniesiona do potomka. Następnie na podstawie tej części tworzona jest mapa definiująca jak elementy genotypu mają być przeniesione do potomka. Można jednocześnie wykonać proces odwrotny do utworzenia dwóch potomków na raz, i jest to zalecane, bo można szybciej utworzyć nową populację. Operator ten obsługuje tylko dwóch rodziców.

Przykład:

$$p_1 = (2 \ 3 \ | \ 1 \ 4 \ 5 \ 8 \ | \ 7 \ 6),$$

$$p_2 = (5 \ 1 \ | \ 2 \ 8 \ 3 \ 4 \ | \ 6 \ 7).$$

Najpierw przenoszone są części pomiędzy punktami cięcia

$$c_1 = (* \ * \ | \ 1 \ 4 \ 5 \ 8 \ | \ * \ *),$$

$$c_2 = (* \ * \ | \ 2 \ 8 \ 3 \ 4 \ | \ * \ *).$$

Na ich podstawie tworzona jest mapa: 1-2, 4-8, 5-3, 8-4. Kolejnym krokiem jest przeniesienie elementów rodzica 2 do potomka 1, oraz rodzica 1 do potomka 2, jeżeli jest to możliwe

$$c_1 = (* \ * \ | \ 1 \ 4 \ 5 \ 8 \ | \ 6 \ 7),$$

$$c_2 = (* \ * \ | \ 2 \ 8 \ 3 \ 4 \ | \ 7 \ 6).$$

Ostatnim krokiem jest dokończenie obu potomków według mapy

$$c_1 = (3 \ 2 \ | \ 1 \ 4 \ 5 \ 8 \ | \ 6 \ 7),$$

$$c_2 = (1 \ 5 \ | \ 2 \ 8 \ 3 \ 4 \ | \ 7 \ 6).$$

[2] [14]

3 Projekt programu

W tym rozdziale zostaną zaprezentowane rezultaty pracy, czyli budowa i działanie zaimplementowanego algorytmu genetycznego. Implementacja opiera się na teorii działania algorytmu Elitist Non-Dominated Sorting Genetic Algorithm. Jest on zgodny z założeniami projektowymi.

3.1 Parametry wejściowe

Algorytm oprócz danych wejściowych opisanych w rozdziale 2.2 przyjmuje parametry wejściowe, które definiują jego konfigurację. Są one przesyłane do głównej metody algorytmu, a użytkownik może je wybrać z poziomu interfejsu użytkownika. Zaimplementowano następujące parametry:

- Rozmiar populacji - liczba całkowita,
- Maksymalna liczba iteracji - liczba całkowita,
- Maksymalna liczba iteracji bez znalezienia lepszego rozwiązania - liczba całkowita,
- Stała liczba iteracji - zmienna logiczna - definiuje, czy algorytm pracuje stałą liczbę iteracji, czy zatrzymuje się gdy nie znajdzie lepszego wyniku,
- Liczba rodziców w krzyżowaniu - liczba całkowita,
- Prawdopodobieństwo mutacji - liczba zmiennoprzecinkowa podwójnej precyzji (zakres od 0 do 1),
- Procent najgorszych osobników poddawanych mutacji - liczba zmiennoprzecinkowa podwójnej precyzji (zakres od 0 do 1),
- Minimalna liczba podmian w mutacji - liczba całkowita,
- Maksymalna liczba podmian w mutacji - liczba całkowita,
- Rodzaj funkcji fitness - obiekt - pozwala wybrać metodę obliczania funkcji fitness (dostępna jest wersja synchroniczna i asynchroniczna współbieżna),
- Rodzaj metody mutacji - obiekt - w praktyce zaimplementowana jest jedna metoda,
- Rodzaj metody krzyżowania - obiekt,
- Rodzaj metody selekcji - obiekt - w praktyce zaimplementowana jest jedna metoda,

- Rodzaj logowania algorytmu - obiekt,
- Log aktywny - zmienna logiczna - definiuje, czy logowanie przebiegu algorytmu jest aktywne,
- Używanie ziarna (seed) - zmienna logiczna - definiuje, czy generowanie populacji podczas inicjalizacji algorytmu przebiega w sposób losowy,
- Ziarno (seed) - liczba całkowita,
- Równoległe obliczanie odległości zagęszczenia - zmienna logiczna - definiuje, czy metoda obliczania odległości zagęszczenia ma być asynchroniczna współbieżna, czy synchroniczna.

3.2 Działanie programu

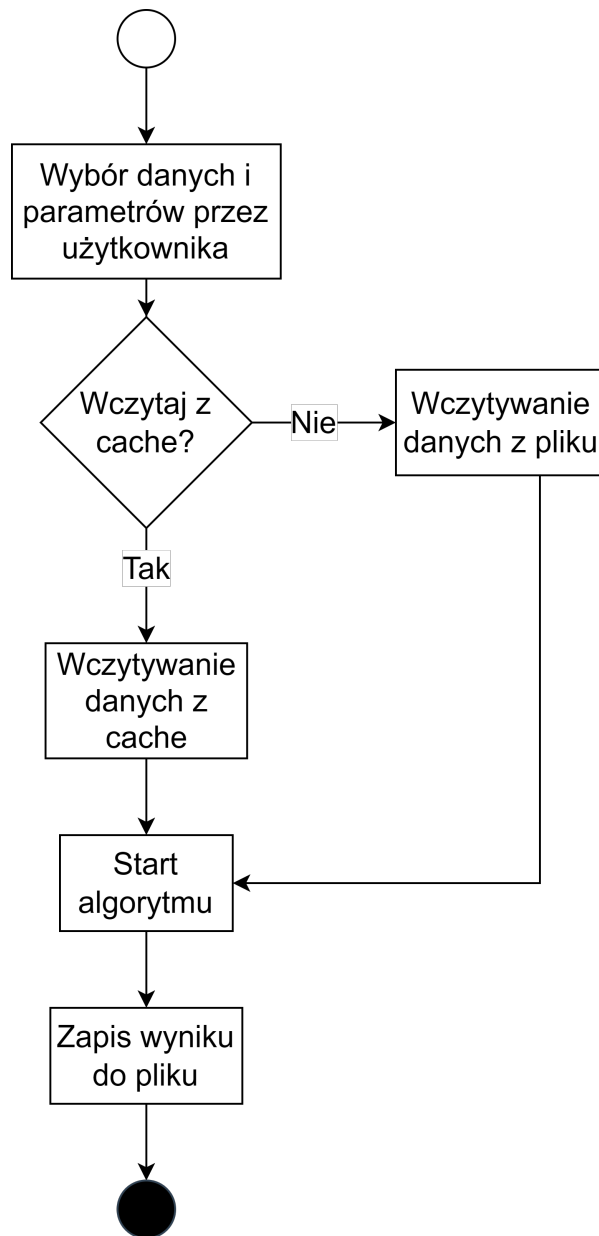
Pełne działanie programu zostało przedstawione na diagramie 3.1. Na początku użytkownik dobiera parametry i dane wejściowe. Dane wejściowe zostały zdefiniowane w rozdziale 2.2. Użytkownik musi przygotować je samodzielnie w dowolnym edytorze plików tekstowych (np. Notepad++) i zapisać w formacie .txt.

Jeżeli cache istnieje, i użytkownik wybiera jego użycie, wtedy dane wejściowe wczytywane będą z cache, co przyspiesza proces inicjalizacji, gdyż nie ma potrzeby wyznaczania takich elementów jak macierze odległości. W przeciwnym wypadku dane wejściowe wczytywane są z plików, a następnie wymagane struktury danych są tworzone przez program. Macierz dystansów zawierająca odległości pomiędzy każdymi dwoma regałami wyznaczana jest za pomocą algorytmu Dijkstry.

Następnie uruchamiany jest algorytm genetyczny z zadanymi parametrami. Po jego ukończeniu zwrócony wynik zapisywany jest do pliku. W tym momencie użytkownik może zamknąć program, lub ponownie skonfigurować i uruchomić algorytm.

3.3 Przebieg algorytmu

Szczegóły na temat algorytmu genetycznego zostały przedstawione na diagramie 3.2. Na początku inicjalizowana jest w sposób losowy populacja o zdefiniowanym przez użytkownika rozmiarze n . Użytkownik alternatywnie może zdecydować o generowaniu populacji na podstawie ziarna (seed). Następnie wyznaczany jest fitness, rangi, oraz odległość zagęszczenia. W tym momencie rozpoczyna się główna pętla. Użytkownik może wybrać dwa warunki terminacji - stałą liczbę iteracji, lub brak poprawy wyniku po zadanej liczbie iteracji.



Rysunek 3.1: Diagram kompletnego programu.

Pierwszym krokiem w głównej pętli jest tworzenie nowej populacji. Pobierane jest r rodziców, i tworzone p potomków, z czego parametry te zależą od wybranych operatorów i parametrów. Pętla selekcji i krzyżowania będzie powtarzała się tak długo, aż utworzona zostanie nowa populacja o rozmiarze n .

Teraz wyznaczany jest fitness nowej populacji. Starej nie ma potrzeby - jest już znany. Następnie obie populacje łączone są w jedną populację o rozmiarze $2n$. Teraz musi zostać wyznaczona ranga i odległość zagęszczenia.

Następnym krokiem jest mutacja. Mutowany jest tylko dolny próg osobników, zależny od dobranego parametru progu w procentach. To gwarantuje, że dobre osobniki w czołówce populacji pozostają nietknięte, a te gorsze osobniki być może zostaną polepszone. Jeżeli zostaną pogorszone nie jest to znaczny problem - najlepsze osobniki pozostają takie same.

Po mutacji niestety trzeba ponownie wyznaczyć fitness, rangi, oraz odległość zagęszczenia. W tym momencie można już pobrać n najlepszych osobników w celu utworzenia nowej populacji używanej w następnej iteracji. Odcięcie połowy populacji wymaga korekty odległości zagęszczenia dla ostatniej rangi - odległość zagęszczenia ściśle zależy od osobników w randze. Jeżeli ilość osobników się zmieni, zmieniają się również poszczególne odległości zagęszczenia.

Teraz pętla powraca do sprawdzenia warunku stopu, i cykl się powtarza. Jeżeli warunek stopu został spełniony, to następuje usunięcie duplikatów, tzn. osobników o identycznym genotypie - duplikaty będą tylko przeszkadzać w doborze rozwiązania spełniającego wymagania w obecnym kontekście. Na końcu osobniki o randze 1, tzn. osobniki optymalne w sensie Pareto są zwracane.

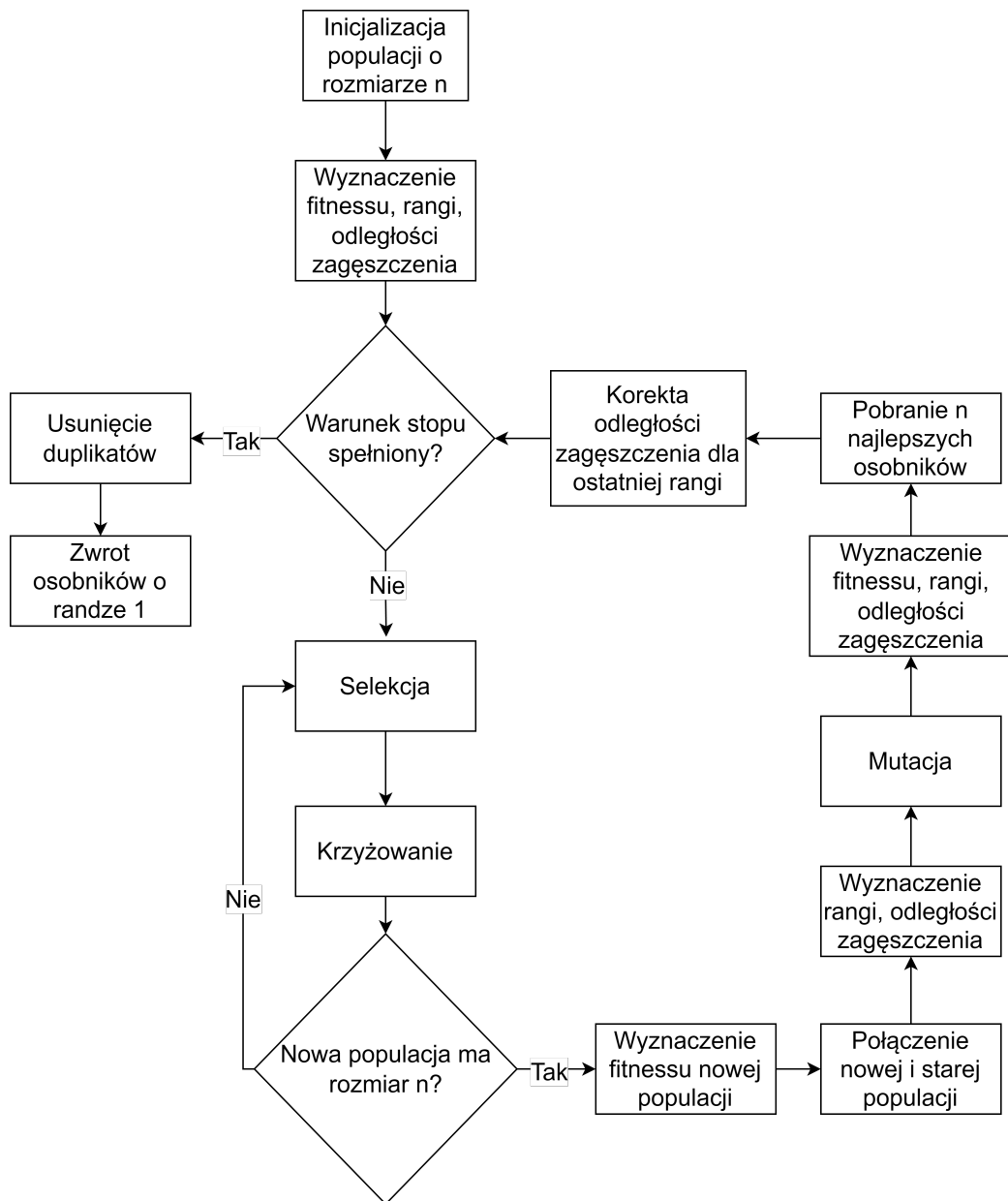
Szczegółowy proces tworzenia nowej populacji przedstawiono na diagramie 3.3. Zakłada się, że osobniki sortowane są malejąco według fitnessu, a konkretnie w przypadku algorytmu wielokryterialnego rosnąco według rangi, a w obrębie rangi malejąco według odległości zagęszczenia. Krótko mówiąc sortowane są od najlepszego do najgorszego osobnika. W kroku 1 istnieje populacja o rozmiarze n . W kroku drugim po selekcji i krzyżowaniu istnieje już populacja o rozmiarze $2n$. W kroku trzecim populacja jest sortowana, aby nie poddać mutacji osobników o wysokiej wartości fitnessu. Następnie populacja jest mutowana, i ponownie obliczane są wartości fitnessu, rangi, i odległości zagęszczenia. Na końcu pobierana jest nowa populacja o rozmiarze n .

3.4 Użyte pakiety

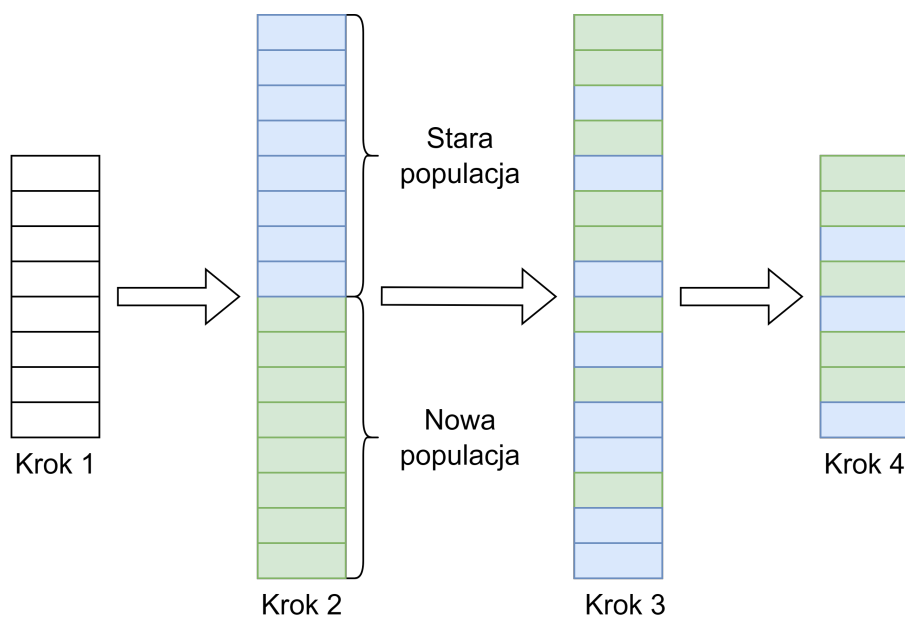
Projekt algorytmu genetycznego nie używa żadnych pakietów, opiera się tylko na możliwościach samej platformy .NET.

Projekt interfejsu użytkownika opiera się na następujących pakietach:

- Avalonia - podstawowy pakiet Avalonia UI,



Rysunek 3.2: Diagram algorytmu genetycznego.



Rysunek 3.3: Diagram tworzenia nowej populacji.

- Avalonia.Controls.DataGrid - dodatkowy pakiet umożliwiający korzystanie z tabeli zwanej Data Grid,
- Avalonia.Desktop - dodatkowy pakiet dedykowany aplikacjom desktopowym,
- Avalonia.Diagnostics - dodatkowy pakiet dostarczający narzędzia do diagnostyki i debugowania aplikacji tworzonych przy pomocy Avalonia UI,
- Avalonia.ReactiveUI - dodatkowy pakiet dostarczający mechanizmy ReactiveUI, tj. responsywnego interfejsu użytkownika korzystającego ze wzorca MVVM.
- FluentAvalonia.ProgressRing - pakiet dodający nowy element interfejsu w postaci pierścienia wczytywania,
- MessageBox.Avalonia - pakiet dodający tzw. message box do Avalonia UI, tj. okna z komunikatem,
- OxyPlot.Avalonia - pakiet pozwalający wyświetlać wykresy w aplikacjach Avalonia,
- ReactiveUI.Fody - pakiet generujący notyfikacje RaisePropertyChange,
- ReactiveUI.Validation - pakiet dodający możliwość walidacji wprowadzanych danych w interfejsie użytkownika,

- XamlNameReferenceGenerator - pakiet generujący referencje do elementów Avalonia UI zadeklarowanych w plikach XAML.

3.5 Struktura kodu

Program składa się z dwóch projektów - algorytmu genetycznego w postaci aplikacji konsolowej, oraz interfejsu użytkownika zaimplementowanego przy użyciu Avalonia UI Framework. Warto zaznaczyć, że Avalonia UI zapewnia kompatybilność zarówno z Windowsem, jak i Linuxem i systemem MacOS. WPF, tj. starsza alternatywa od Microsoftu działa tylko pod Windowsem, dlatego nie został tutaj wykorzystany. Projekt konsolowy jest niezależny i może działać sam. Projekt desktopowy korzysta z projektu konsolowego, zawiera odniesienia do metod algorytmu genetycznego (przede wszystkim głównej pętli algorytmu). Takie podejście jest lepsze niż projektowanie aplikacji jako jednego projektu, gdyż w razie potrzeby można kod łatwo wykorzystać w innym celu, np. utworzyć API, albo zmienić platformę (np. z desktopowej na webową).

Zastosowano dwa wzorce projektowe:

- Factory - pozwala na podmianę klas podczas działania programu - to z kolei pozwala na zmianę operatorów algorytmu z perspektywy interfejsu użytkownika, bez potrzeby pisania nadmiarowego kodu.
- MVVM - Model-View-ViewModel - definiuje sposób tworzenia aplikacji desktopowych tak, aby powiązanie interfejsu z kodem backendowym było jak najmniejsze. To pozwala na łatwą zmianę technologii gdy zachodzi taka potrzeba, gdyż ten sam kod backendowy można wykorzystać w wielu różnych projektach.

Struktura projektu jest bardzo złożona, dlatego zostanie zaprezentowana w sposób uproszczony:

- Abstract - abstrakcyjne klasy potrzebne do implementacji wzorca Factory, który pozwala na podmianę klas w czasie działania programu. To pozwala na przełączanie się pomiędzy operatorami z perspektywy interfejsu użytkownika.
- DataHandler - metody obróbki danych - wczytywanie danych, zapis danych, i konwersja na format czytelny dla aplikacji.
- Loggers - klasy obsługi logów.
- Logic - klasy z metodami pomocniczymi, głównie dla obliczania fitnessu, rangi, i odległości zagęszczenia.
- Models - modele danych.

- Operators - operatory algorytmu.
- Other - wszystkie inne metody. Zawiera: obsługę cache, kopiowanie głębokie (kopiowanie danych tak, aby pozbyć się referencji, to pozwala na edycję kopii bez edycji oryginału), rozszerzenia, fabrykę (obsługa wzorca Factory), oraz metody obliczania długości ścieżek (potrzebne dla utworzenia macierzy dystansów).
- Static - statyczne metody, głównie używane do udostępniania danych potrzebnych dla algorytmu w wielu miejscach (np. macierz dystansów).
- GeneticAlgorithm - zawiera główną pętlę algorytmu i metody pomocniczne.
- Program - zawiera metodę main potrzebną do uruchomienia aplikacji bez interfejsu użytkownika.

Struktura aplikacji desktopowej z interfejsem użytkownika prezentuje się następująco:

- Models - modele danych.
- Other - zawiera obsługę cache interfejsu użytkownika.
- Styles - zawiera style interfejsu użytkownika.
- ViewModels - zawiera view modele, tzn. logikę biznesową interfejsu użytkownika.
- Views - zawiera widoki, tj. okna interfejsu użytkownika.
- Program - zawiera punkt wejściowy programu (Avalonia AppBuilder).

Istnieje również trzeci projekt z testami jednostkowymi. Zostanie on omówiony później w rozdziale 3.10.

3.6 Elementy i operatory algorytmu

Poniżej konkretnie opisano zaimplementowane operatory. Kod źródłowy nie zostanie przedstawiony ze względu na jego złożoność i długość, ale metody zostaną zwięźle zaprezentowane za pomocą pseudokodów.

3.6.1 Fitness

Implementacja fitnessu to element najbardziej oczywisty. W tym wypadku dostępny jest opis matematyczny, wystarczy te wzory zaimplementować. Implementacje opisano pseudokodem 3.1. Trzeba jednak doprecyzować, jak obliczany jest dystans produktów między sobą. Jeżeli oba towary są na tym samym regale, ich dystans wynosi 0, ale nie jest to dobre podejście. Algorytm faworyzowałby układy gdzie związane towary są na tych samych regałach, co z kolei prowadzi do tego, że trzeba byłoby sięgać na wysokie półki, a o wiele łatwiej jest po prostu zabrać towar z niskiego poziomu na regale obok. Dlatego jeżeli towary są na tym samym regale, przydzielany jest dystans

$$abs(h_1 - h_2) + 1.$$

Litera h oznacza wysokość (piętro) towaru na regale. To gwarantuje, że lepiej jest położyć związany towar na regale obok, niż układać wszystko na jednym regale. Istnieje ryzyko, że związany towar zostanie położony wysoko na innym regale, ale to powinno zostać obsłużone przez inne kryteria.

Istnieje jeszcze drugi wariant operatora fitness, a konkretnie wersja równoległa. Nie różni się niczym od głównego operatora poza tym, że fitness obliczany jest współbieżnie dla każdego osobnika.

Algorytm 3.1 Fitness

```
for all individuals do
  fitness = 0
  for all elements in genotype do
    fPopularity = fPopularity +
      (popularity * distanceFromExit + popularity * level)
    fWeight = fWeight +
      (weight * distanceFromExit + weight * level)
    for all orders do
      if element does not exist in order then continue
      end if
      fDistances = fDistances +
        orderCount * dist(element, otherProducts)
    end for
  end for
end for
```

3.6.2 Ranga i odległość zagęszczenia

Wyznaczanie rangi i odległości zagęszczenia przeprowadzane jest po kolei - najpierw nadawane są rangi, a następnie w każdej randze wyznaczane są odległości zagęszczenia. Wyznaczanie rangi przedstawiono za pomocą pseudokodu 3.2. Wyznaczanie odległości zagęszczenia za pomocą prostopadłościaków prezentuje pseudokod 3.3. Realna implementacja uwzględnia sprawdzenie czy nie występuje dzielenie przez zero - pominięto to w pseudokodzie. Odległość zagęszczenia to jedna wartość uwzględniająca wszystkie trzy kryteria. Jako, że kryteria te są niezależne i często o różnej skali, bez normalizacji wartości te będą mało miarodajne, gdyż jedno kryterium może mocno zdominować inne kryteria. Dlatego normalizacja jest konieczna aby uniknąć tzw. stroniczości kierunkowej.

Użytkownik może zdecydować o włączeniu trybu równoległego dla obliczania odległości zagęszczenia. Wtedy wartości te będą obliczane współbieżnie dla każdej rangi.

Algorytm 3.2 Wyznaczanie rangi

```
set all ranks to 0
notRanked = population
currentRank = 1
while notRanked.Count > 0 do
    for all not ranked individuals do
        if individual not dominated by other not ranked individuals then
            individual.Rank = currentRank
        end if
    end for
    remove ranked individuals from notRanked
    currentRank++
end while
```

3.6.3 Selekcja

Zgodnie z założeniami zaimplementowano Crowded Tournament Selection uwzględniającą rangi i odległości zagęszczenia. Jej działanie jest następujące - wybiera się dwa osobniki, i zwracany jest ten z mniejszą rangą. Gdy rangi są równe, zwracany jest ten z większą odległością zagęszczenia. Gdy obie wartości są równe można zwrócić losowego osobnika, gdyż prawdopodobieństwo takiego wydarzenia jest stosunkowo niskie. Implementację przedstawiono pseudokodem 3.4. Zakłada ona możliwość zwrócenia więcej niż jednego osobnika. Zmienna *selectionSize* w tym kontekście oznacza liczbę rodziców, trzeba wybrać tyle osobników, aby można było przeprowadzić operację krzyżowania.

Algorytm 3.3 Wyznaczanie odległości zagęszczenia

```
set all crowding distances to 0
for all rank in population do
    order by fPopularity
    fmax = fPopularity.Max()
    fmin = fPopularity.Min()
    for i = 0; i < individualsInRank; i++ do
        if first or last then crowdingDistance = PositiveInfinity
        else crowdingDistance = crowdingDistance +
            (fPopularity[i+1] - fPopularity[i-1]) / (fmax - fmin)
        end if
    end for
    order by fWeight
    fmax = fWeight.Max()
    fmin = fWeight.Min()
    for i = 0; i < individualsInRank; i++ do
        if first or last then crowdingDistance = PositiveInfinity
        else crowdingDistance = crowdingDistance +
            (fWeight[i+1] - fWeight[i-1]) / (fmax - fmin)
        end if
    end for
    order by fDistances
    fmax = fDistances.Max()
    fmin = fDistances.Min()
    for i = 0; i < individualsInRank; i++ do
        if first or last then crowdingDistance = PositiveInfinity
        else crowdingDistance = crowdingDistance +
            (fDistances[i+1] - fDistances[i-1]) / (fmax - fmin)
        end if
    end for
end for
```

Algorytm 3.4 Selekcja turniejowa

```
selection = population.takeRandom(selectionSize*2)
for i = 0; i < selection.Count; i+=2 do
    candidates = selection.Skip(i).Take(2)
    winner = performTournament(candidates)
    selected.Add(winner)
end for
return selected
#list of candidates must contain 2 elements
def performTournament(list candidates)
    if candidates[0].rank < candidates[1].rank then
        return candidates[0]
    else if candidates[0].rank == candidates[1].rank then
        if candidates[0].crowdingDist > candidates[1].crowdingDist then
            return candidates[0]
        else return candidates[1]
    end if
else return candidates[1]
end if
```

3.6.4 Mutacja

Zgodnie z założeniami zaimplementowano mutację która podmienia losowe elementy genotypu. Jej działanie przedstawiono za pomocą pseudokodu 3.5. Od razu warto zaznaczyć, że nie trzeba mutacji poddawać całej populacji, do metody można przekazać tylko wybrany zakres. Parametry *probability*, *minSwaps*, oraz *maxSwaps* zadawane są przez użytkownika. Logika działania jest następująca - pętla iteruje przez każdego osobnika, i jeżeli wylosowana liczba jest mniejsza od zadanego prawdopodobieństwa, to dany osobnik poddawany jest mutacji. Tutaj mutacją jest podmiana losowej ilości elementów genotypu. Istnieje pewne ryzyko, że podmieniane będą te same elementy, co z kolei doprowadzi do praktycznie zerowej mutacji, ale ryzyko takiego wydarzenia można uznać za wystarczająco małe, aby było akceptowalne.

3.6.5 Krzyżowanie AEX

Zaimplementowano operatory krzyżowania zgodnie z założeniami przedstawionymi w rozdziale 2.4.3. Implementację operatora AEX przedstawiono za pomocą pseudokodu 3.6. Realna implementacja jest stosunkowo skomplikowana, dlatego została tutaj zdefiniowana mocno opisowo, ale logika jej działania jest

Algorytm 3.5 Mutacja

```
for all individuals in population do
  if random.NextDouble() < probability then
    var swapsCount = random.Next(minSwaps, maxSwaps+1)
    var currentSwaps = 0
    while currentSwaps < swapsCount do
      var targets = GetTwoRandomElementIDsFromGenotype()
      var first = targets[0]
      var second = targets[1]
      var b = individual.Genotype[first]
      individual.Genotype[first] = individual.Genotype[second]
      individual.Genotype[second] = b
      currentSwaps++
    end while
  end if
end for
return population
```

identyczna.

3.6.6 Krzyżowanie ERX

Implementację operatora ERX przedstawiono za pomocą pseudokodu 3.7. Aby efektywnie odczytywać liczbę sąsiadów tworzona jest macierz sąsiedztwa. Na jej podstawie wykonywana jest większość obliczeń. Następnie zgodnie z założeniami ERX w każdej iteracji pobierany jest sąsiadujący element z najmniejszą liczbą sąsiadów.

Teoria operatora ERX zakłada, że zawsze taką metodą można dobrać następny element. W praktyce może wystąpić sytuacja, że wszyscy dozwoleni do doboru sąsiedzi istnieją już w potomku. Proponowanym tutaj rozwiązaniem tego problemu jest wybranie losowego elementu nieistniejącego jeszcze w potomku z pierwszego rodzica. Wybór rodzica jest obojętny, bo obaj rodzice mają takie same elementy w genotypie, tylko w innej kolejności.

3.6.7 Krzyżowanie PMX

Implementacja operatora PMX została przedstawiona za pomocą pseudokodu 3.8. Operator PMX zakłada istnienie pojedynczej mapy. Tutaj proponowane jest utworzenie dwóch map, po jednej dla każdego potomka. Wynika to z działania języka C# który nie pozwala na bezpośredni odczyt klucza według wartości.

Algorytm 3.6 Operator AEX

```
currentParent = 0
#get first arc from current parent
offspring.AddRange(parents[currentParent].GetFirstArc())
currentParent++
while offspring not complete do
    if feasible arc exists in current parent then
        #get next arc from current parent
        offspring.AddRange(parents[currentParent].GetNextArc())
    else
        #get random feasible arc
        offspring.AddRange(parents[currentParent]
            .GetNextArc(random = true))
    end if
    if currentParent is not last then
        currentParent++
    else currentParent = 0
    end if
end while
return offspring
```

Algorytm 3.7 Operator ERX

```
matrix = ConstructAdjacencyMatrix(parents)
offspring[0] = parents[0].Genotype[0]
for i = 1; i < parents[0].Genotype.Count; i++ do
    neighbours = GetNeighboursOfCurrentElement(matrix, offspring.Last())
    offspring[i] = GetNeighbourWithTheLeastNeighbours(matrix, neighbours)
    #returns -1 if feasible neighbour couldn't be found
    if offspring[i] == -1 then
        offspring[i] = GetRandomElement(offspring, parents[0].Genotype)
        #get random element not present in offspring
    end if
end for
return offspring
```

Utworzenie drugiej mapy analogicznie do pierwszej będzie efektywniejsze niż próba odwrócenia jej. Konkretnie użyto słownika, który jest implementacją mapy w .NET.

Kolejną zmianą względem teorii działania operatora PMX jest nieco inny odczyt z mapy. Przykładowo dla mapy 3 -> 2, 4 -> 3, 1 -> 5 gdy odpowiadający element rodzica 2 dla potomka 1 to 4, należy przypisać element 3. Jeżeli element 3 już istnieje, algorytm przestaje działać poprawnie. Proponowanym wyjściem z tej sytuacji jest odczytywanie wartości z mapy w pętli. Jeżeli wybrany element jest duplikatem, należy użyć go do odczytu w celu wybrania nowego elementu. Okazuje się, że takim sposobem zawsze można dobrać odpowiedni element, i operacja nigdy nie wpadnie w nieskończoną pętlę.

Algorytm 3.8 Operator PMX

```
cutPoints = GetTwoRandomCutPoints(parentsLength)
CopyElementsBetweenCutPoints(offspring1, parent1, cutPoints)
CopyElementsBetweenCutPoints(offspring2, parent2, cutPoints)
mapO1 = CreateMap(offspring1, offspring2)
mapO2 = CreateMap(offspring2, offspring1)
#if offspring does not contain them already - copy
CopyElementsFromOppositeParent(offspring1, parent2)
CopyElementsFromOppositeParent(offspring2, parent1)
FillValuesAccordingToMap(offspring1, parent2, map1)
FillValuesAccordingToMap(offspring2, parent1, map2)
offspring.Add(offspring1, offspring2)
return offspring
```

3.7 Wyznaczanie rozwiązania najlepszego oraz współczynnik poprawy

Algorytm genetyczny wielokryterialny zwraca wiele wyników optymalnych w sensie Pareto. Osoba podejmująca decyzje może wybrać które rozwiązanie jest najlepsze w danym momencie według ważności kryteriów. W obecnym przypadku ważne są wszystkie trzy kryteria. Aby umożliwić wybranie rozwiązania najlepszego z zestawu rozwiązań optymalnych wprowadzono dodatkową wartość nazwaną znormalizowaną sumą. Dla każdego osobnika wyznaczana jest suma znormalizowanych kryteriów. Algorytm przedstawiono pseudokodem 3.9. Osobnik z najmniejszą sumą najlepiej spełnia wszystkie trzy kryteria.

Suma ta jest wyznaczana dopiero po zakończeniu działania algorytmu dla znalezionej zbioru rozwiązań i nie wpływa na działanie algorytmu. Jest to

tylko dodatkowa pomoc do wyznaczenia jednego rozwiązania które może zostać uwzględnione w ocenie efektywności algorytmu.

Algorytm 3.9 Wyznaczanie znormalizowanej sumy

```
#sum of all fitness values for each criterion
popSum = individuals.Sum(FitnessPopularity)
weightSum = individuals.Sum(FitnessWeight)
distSum = individuals.Sum(FitnessDistances)
for i = 0; i < individuals.Count; i++ do
    individuals[i].NormalizedPopularity =
        individuals[i].FitnessPopularity / popSum
    individuals[i].NormalizedWeight =
        individuals[i].FitnessWeight / weightSum
    individuals[i].NormalizedDistances =
        individuals[i].FitnessDistances / distSum
    individuals[i].NormalizedSum =
        individuals[i].NormalizedPopularity +
        individuals[i].NormalizedWeight +
        individuals[i].NormalizedDistances
end for
```

Wprowadzono również wartość nazwaną średnią poprawą. W pierwszej (zerowej) iteracji za pomocą znormalizowanej sumy wyznaczany jest osobnik najlepiej spełniający wszystkie kryteria, i jest zapisywany jako najlepszy pierwszy osobnik. Po zakończeniu działania algorytmu i zwróceniu wyników, dla każdego osobnika obliczane jest o ile procent każde z kryteriów zostało zmniejszone względem pierwszego najlepszego osobnika, i dla każdego wyniku wyznaczana jest następnie średnia poprawa. Algorytm wyznaczania przedstawiono pseudokodem 3.10.

Istnieje możliwość, że algorytm znajdzie bardzo dobry wynik w iteracji pierwszej po prostu w sposób losowy. Wtedy istnieje ryzyko, że średnia poprawa będzie negatywna - algorytm będzie już tylko błądził wokół dobrych rozwiązań bez dużej poprawy. Jeżeli nawet jedno kryterium zostanie mocno pogorszone, może to wpłynąć negatywnie na wartość średniej poprawy mimo, że wynik z definicji jest optymalny. Nie jest to błąd implementacji, tylko pewna anomalia. Ma bardzo niskie szanse na wystąpienie, większe dla problemów bardzo małych (np. 6 towarów), gdzie znalezienie wyniku poprzez pierwsze losowe permutacje jest bardzo proste, aczkolwiek w realnych warunkach takie problemy nie będą rozpatrywane.

W praktyce w puli rozwiązań mogą znajdować się wyniki, gdzie pojedyncze kryteria zostały mocno pogorszone, co z kolei prowadzi do negatywnej średniej

poprawy. Celem jest jednak znalezienie rozwiązań które dobrze spełniają wszystkie kryteria, dlatego taki system oceny jakości rozwiązań wydaje się być akceptowalny.

Algorytm 3.10 Wyznaczanie średniej poprawy

```
for i = 0; i < individuals.Count; i++ do
    individuals[i].PopularityImprovement =
        InitialBestFitnessPopularity - individuals[i].FitnessPopularity;
    individuals[i].WeightImprovement =
        InitialBestFitnessWeight - individuals[i].FitnessWeight;
    individuals[i].DistancesImprovement =
        InitialBestFitnessDistances - individuals[i].FitnessDistances;
    individuals[i].AverageImprovement =
        (individuals[i].PopularityImprovement /
         InitialBestFitnessPopularity +
         individuals[i].WeightImprovement /
         initialBestFitnessWeight +
         individuals[i].DistancesImprovement /
         initialBestFitnessDistances)
        / 3 * 100;
end for
```

Należy również wspomnieć, że średnia poprawa nigdy nie osiągnie 100%, wynika to z matematycznych definicji kryteriów oceny. Funkcje te nawet dla optymalnych układów nie będą mieć wartości 0. Dla kryterium odległości między towarami nie jest to tak widoczne, bo odległości te mogą spaść bardzo mocno, ale jest to bardziej dotkliwe dla kryteriów popularności i wagi. Aby wyznaczyć maksymalną możliwą poprawę należałoby wyznaczyć funkcję która znajduje rozwiązanie optymalne dla jednego kryterium, ale nie jest to celem tej pracy. Użyto swego rodzaju metody zachłannej, gdzie po prostu posortowano towary względem popularności. Z dokładnością do liczb całkowitych maksymalna możliwa poprawa wynosi 55%. Wartość dokładna nie powinna różnić się zbyt wiele, dlatego można uznać, że maksymalna możliwa poprawa kryteriów popularności i wagi to 50-60%.

To prowadzi do pewnych wniosków. Jeżeli po inicjalizacji populacji osobniki startują już z poprawą względem rozwiązania najgorszego równą np. 30%, to poprawa 10% lub 20% mimo, że wydaje się niska, jest tak na prawdę bardzo dobrym wynikiem. To oznacza, że algorytm nie będzie zawsze miał zbyt wysokich wartości poprawy pierwszych dwóch kryteriów, nawet jeżeli będzie efektywny.

3.8 Interfejs użytkownika

Interfejs użytkownika został skupiony w jednym oknie w celu ułatwienia korzystania z programu. Zawiera trzy główne strefy - sekcję danych wejściowych, gdzie użytkownik wybiera pliki z danymi, sekcję parametrów, gdzie użytkownik dobiera parametry algorytmu, oraz sekcję wyników - czas działania, najlepsze osobniki (rozwiązania), oraz wykres fitnessu.

Lista rozwiązań (osobników o randze 1) zawiera identyfikator, numer iteracji znalezienia rozwiązania, wartości kryteriów, sumę znormalizowaną, średnią poprawę, oraz poprawę każdego kryterium. Wykres fitnessu rysowany jest w czasie rzeczywistym podczas działania programu. Interfejs użytkownika przedstawiono na rzucie ekranu 3.4 i 3.5. Podczas działania algorytmu interfejs informuje użytkownika za pomocą tzw. spinnera, że algorytm pracuje, co przedstawiono na rysunku 3.6.

3.9 Logi i dane wyjściowe

Przebieg algorytmu jest logowany. Dostępne są trzy opcje - do pliku, do pliku w wersji zminimalizowanej, lub do konsoli. Format logów w konsoli oraz w pliku nie różni się. W przypadku wybrania logowania do konsoli uruchamia się okno konsoli w którym wyświetlany będzie log. Dane wyjściowe zapisywane są tylko do pliku. Zawierają informacje o konfiguracji algorytmu dla danego przebiegu, statystyki działania algorytmu, oraz konkretne znalezione układy towaru w magazynie. Inne logi służą głównie analizie samego przebiegu algorytmu i zachowania populacji osobników. Log w wersji zminimalizowanej jest zaprojektowany z myślą o wizualizowaniu danych na ręcznie robionych wykresach, np. za pomocą języka Python, czy w Excelu.

Logi wyświetlane w konsoli zawierają numer iteracji, średnie wartości kryteriów oraz ich odchylenia standardowe, oraz listę osobników w każdej iteracji. Każdy osobnik zawiera:

- Identyfikator,
- Rangę i odległość zagęszczenia,
- Wartości kryteriów,
- Genotyp.

Logi zapisywane do pliku zawierają identyczne informacje.

Log w wersji zminimalizowanej ma format .csv i zawiera następujące kolumny:

Input

Read matrix

Off

Load products file

G:\...\Magister\Dane\WB1csv.txt

Load warehouse file

G:\...\Magister\Dane\WB1.txt

Load matrix file

C:\...\net6.0\input\WarehouseStructureMatrix.txt

Load racks file

C:\...\net6.0\input\WarehouseStructureMatrixRS.txt

Load orders file

G:\...\Magister\Dane\WB1o.txt

Use cache

Off

Create cache

On

Cache name

Cache1

Select cache

Select cache

Parameters

Fitness

Default

Crossover

PMX

Mutation

Swap

Selection

Tournament

Enable log

Off

Log

FileMini

Population size

100

Use seed

Off

Seed

5

Hard stop

On

Max iterations

50

Max iterations without improvement

40

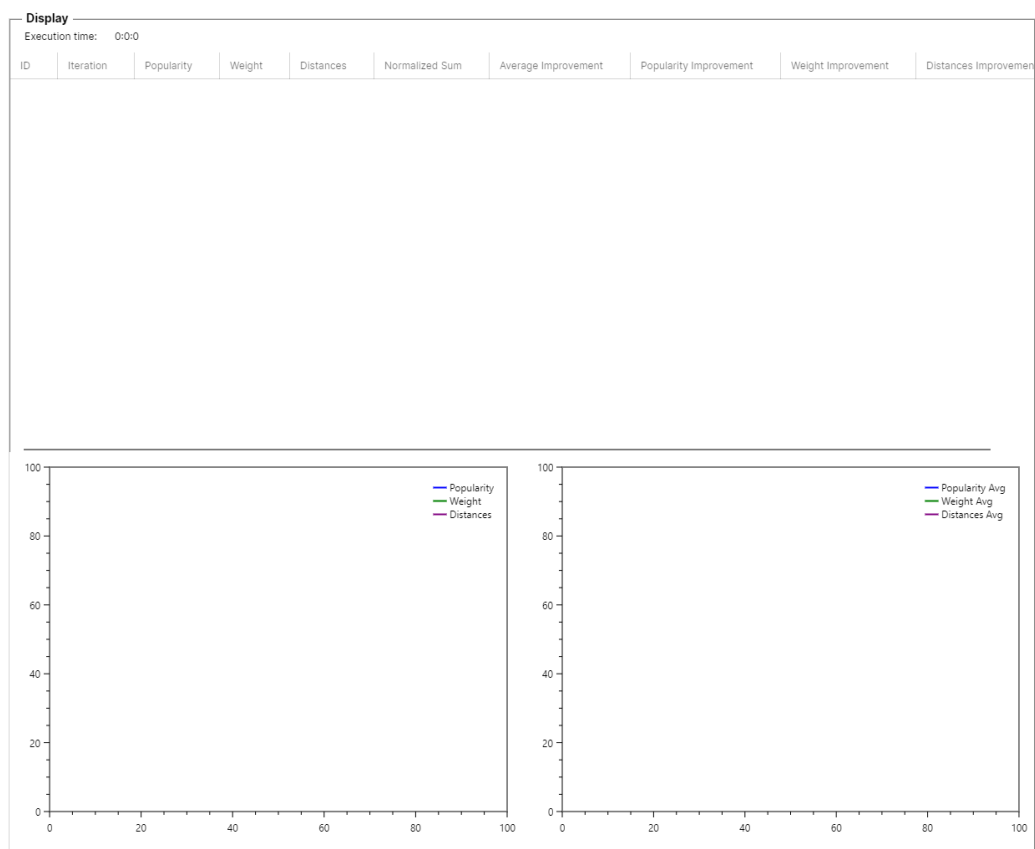
Parents count

2

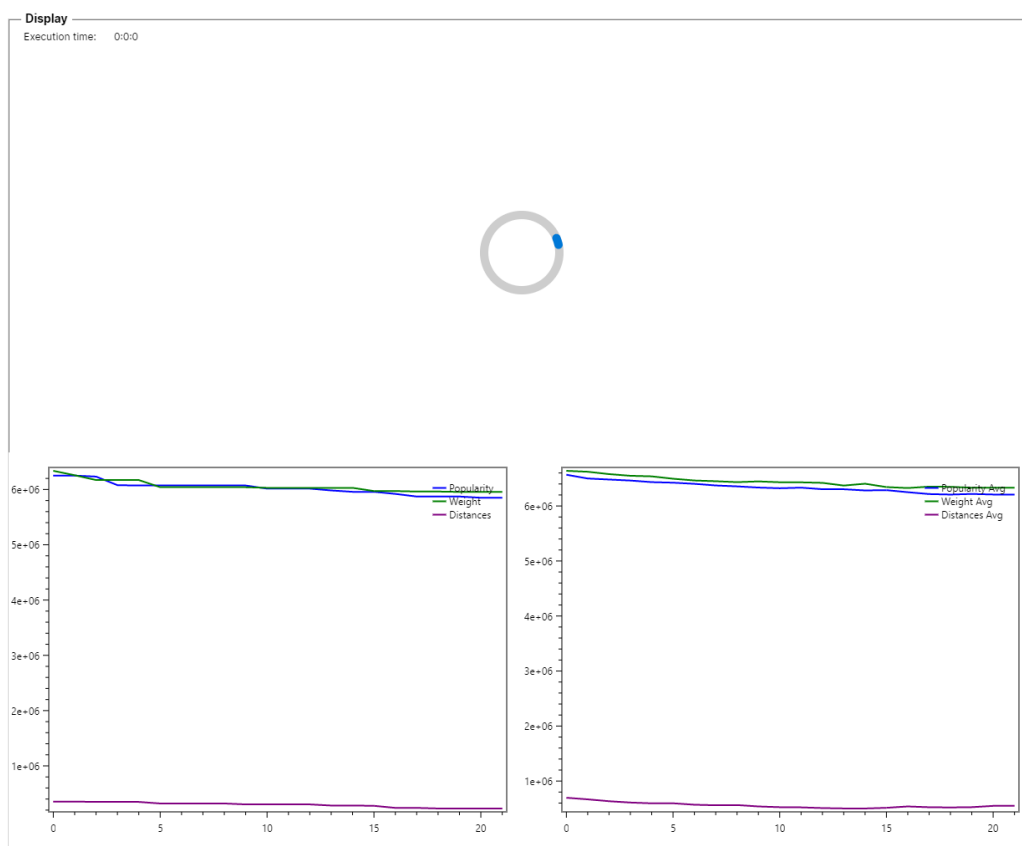
Start

Stop

Rysunek 3.4: Interfejs użytkownika - lewy panel.



Rysunek 3.5: Interfejs użytkownika - prawy panel.



Rysunek 3.6: Interfejs użytkownika podczas działania algorytmu.

- Numer iteracji,
- Minimum każdego kryterium w danej iteracji,
- Średnią każdego kryterium,
- Odchylenie standardowe każdego kryterium.

Plik tekstowy z danymi wyjściowymi skupia się na podsumowaniu przebiegu i znalezionych rezultatach. Zawiera:

- Ustawione parametry wejściowe,
- Czas pracy,
- Proponowany najlepszy wynik (zgodnie z rozdziałem 3.7),
- Średnia i odchylenie standardowe kryteriów w ostatniej iteracji,
- Wszystkie znalezione wyniki w randze pierwszej, wraz z konkretnymi układami towarów w magazynie.

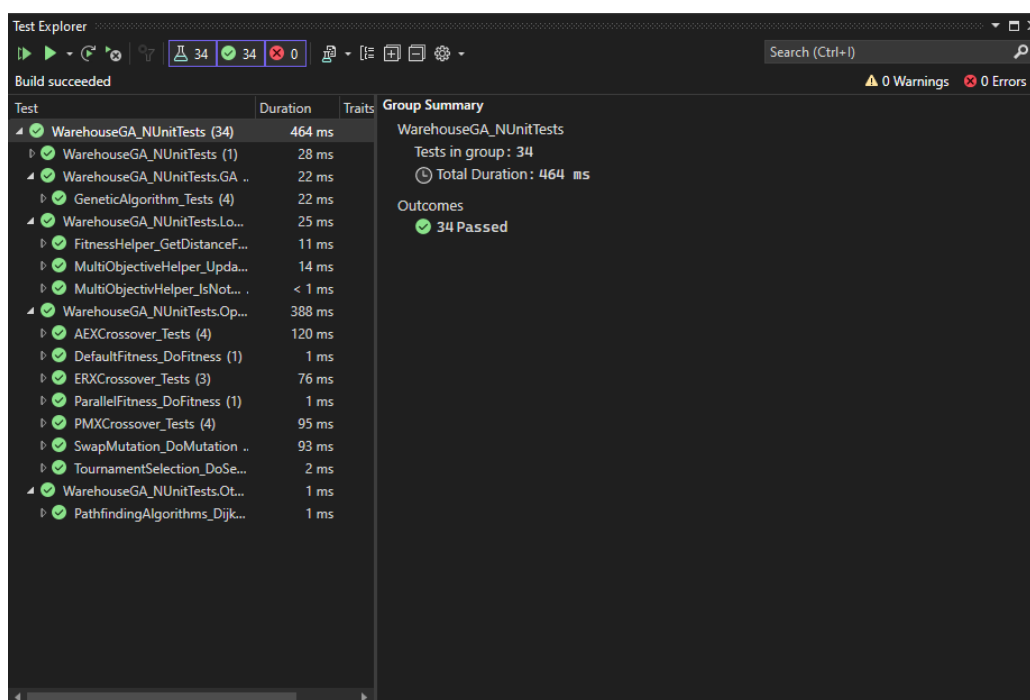
3.10 Testy programu

Program był testowany manualnie, ale również za pomocą automatycznych testów jednostkowych. W testach jednostkowych uwzględniono przede wszystkim kluczowe elementy algorytmu. Testy manualne składały się z debugowania w środowisku programistycznym i podglądzie czy wszystkie wartości są w przewidywanym zakresie, oraz analizie logów. W automatycznych testach jednostkowych uwzględniono:

- Fitness,
- Metody pomocnicze dla fitnessu,
- Metody obliczania rangi i odległości zagęszczenia,
- Metodę wyznaczania czy osobnik jest zdominowany,
- Algorytm wyznaczania najkrótszych ścieżek - algorytm Dijkstry,
- Metody wyznaczania poprawy wyniku,
- Krzyżowanie AEX,
- Krzyżowanie PMX,

- Krzyżowanie ERX,
- Mutację,
- Selekcję.

W przypadkach gdzie istnieją sztywne matematyczne metody wyznaczania wartości (np. fitness), w testach tworzone scenariusze wyznaczania tych wartości, i sprawdzano czy są one równe. W przypadkach gdzie istnieje duża losowość i nie da się wyznaczyć dokładnej wartości (np. krzyżowanie AEX), sprawdzano tylko czy wartość jest wiarygodna. Testowano również czy nie ma innych rodzajów błędów takich jak problemy z referencjami (zmiana jednej wartości zmienia również inną wartość). Część testów, w szczególności te z dużą losowością, powtarzano w pętli po kilkaset razy, aby mieć pewność, że faktycznie wszystko jest w normie. Zrzut ekranu prezentujący eksplorator testów przedstawiono na rysunku 3.7



Rysunek 3.7: Testy jednostkowe.

3.11 Badanie wydajności programu

W tym rozdziale zostanie zbadana wydajność programu, a konkretnie użycie procesora i pamięci RAM. Do badań użyto oprogramowania JetBrains dotMemory

oraz JetBrains dotTrace. Badania przeprowadzono na magazynie ze 132 towarami, co powinno zapewnić wystarczające obciążenie systemu. Przetestowane będą wszystkie trzy operatory krzyżowania, z tymi samymi parametrami:

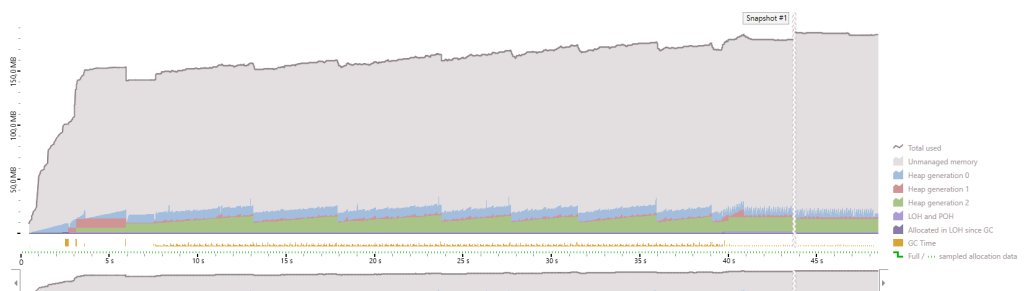
- Fitness - synchroniczny,
- Rozmiar populacji - 100,
- Stały seed populacji - 5525,
- Liczba iteracji - 100,
- Liczba rodziców - 2,
- Prawdopodobieństwo mutacji - 80%,
- Mutacja osobników - 60%,
- Liczba podmian w mutacji - od 30 do 60,
- Odległość zagęszczenia - synchroniczna.

3.11.1 Użycie pamięci RAM

Na rysunku 3.8 pokazano rezultat dla operatora AEX. Linia czasu dla operatorów ERX oraz PMX wygląda praktycznie identycznie - występują ciągle wzrosty i spadki. Najprawdopodobniej spadkiem są momenty finalizowania iteracji i przejście do następnej. Nasuwa się jednak jeden wniosek - rozmiar generacji drugiej na stercie cyklicznie rośnie i spada. Operacja przenoszenia z generacji 1 do generacji 2 może być kosztowna, dlatego można się zastanowić nad możliwościami optymalizacji kodu pod tym kątem.

Warto jeszcze spojrzeć na czas Garbage Collector. Jak widać uruchamia się on bardzo często. Dla operatorów ERX i PMX wygląda bardzo podobnie, tylko punkty maksymalne są jeszcze wyższe. Widoczne są również punkty, w których Garbage Collector nie pracuje, gdzie dla AEX praktycznie w każdym momencie jest on uruchamiany. Dla tego problemu ciężko mówić o nieakceptowalnym narzucie czasowym, ale dla bardzo dużych problemów, gdzie czas pracy algorytmu będzie znacznie dłuższy, można zastanowić się nad przełączeniem trybu Garbage Collector do trybu Server. W trybie Server Garbage Collector dopuszcza większy pobór pamięci, ale uruchamia się rzadziej. Domyślny tryb to tryb Workstation.

Podgląd elementów w pamięci RAM pokazuje, że najwięcej miejsca zajmuje schemat magazynu, lista towarów powiązanych, oraz lista osobników, w dokładnie takiej kolejności. Również dużo miejsca zajmują pewne elementy systemowe, ale na to programista ma mały wpływ.



Rysunek 3.8: Okno programu dotMemory - użycie pamięci RAM dla AEX.

W tabeli 3.1 pokazano rezultaty pomiaru w snapshocie robionym na samym końcu działania algorytmu. Wyniki nie różnią się zbytnio w zależności od operatora krzyżowania, dlatego wyniki przedstawiono w jednej tabeli.

Element	RAM
Całkowite	192 MB +- 8 MB
.NET całkowite	15 MB +- 2 MB
.NET używane	15 MB +- 2 MB

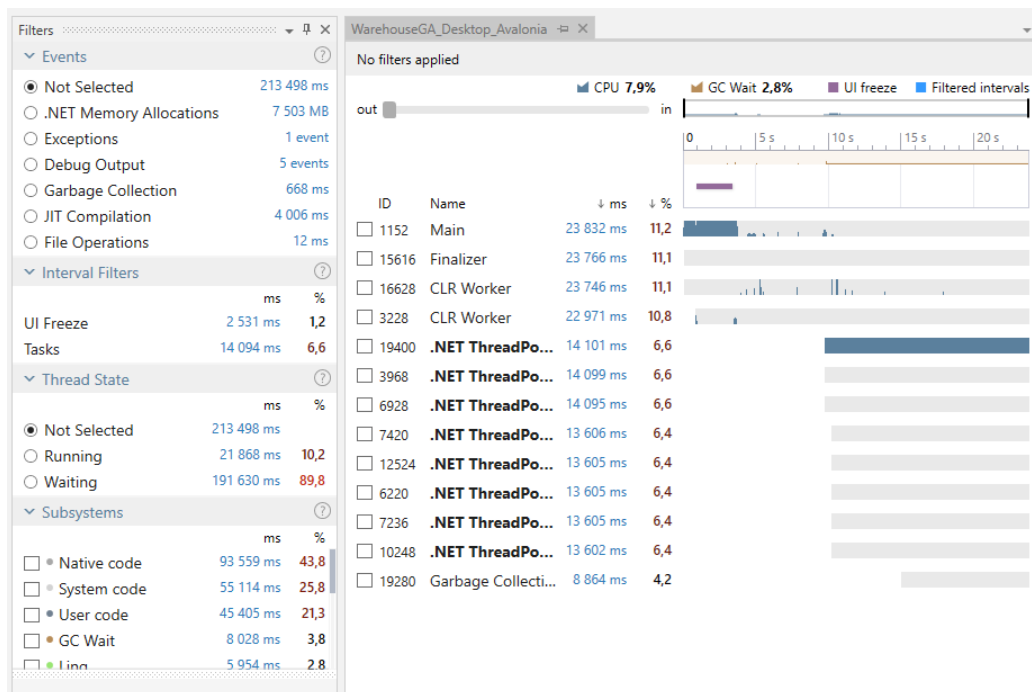
Tabela 3.1: Użycie pamięci RAM.

Pomiary wykazują, że wpływ operatorów krzyżowania na użycie pamięci jest minimalne. Dużo również zależy od momentu w którym robiony jest snapshot. Bezpiecznie jest więc stwierdzić, że faktyczny wpływ operatorów krzyżowania na użycie pamięci jest pomijalnie małe. Jedynie na co mają wpływ, to na czas pracy Garbage Collector, gdzie najgorzej wypada operator AEX, ale to może być kwestia konkretnej implementacji, a nie obiektywnej jakości operatora.

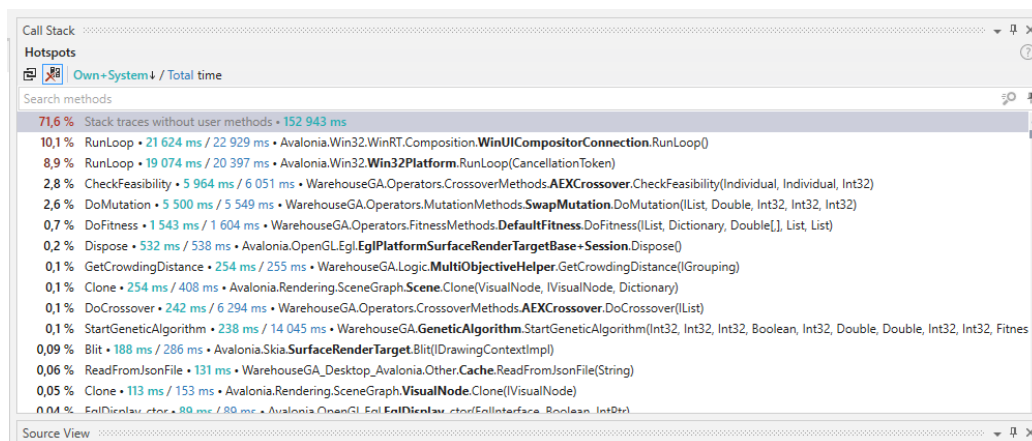
Podsumowując, można uznać, że użycie pamięci RAM jest w granicach akceptacji. Oczywiście dla większych danych będzie większe, ale źle napisany kod może bardzo negatywnie wpłynąć na użycie pamięci nawet dla małych danych. Nie przeprowadzono testów jaka jest złożoność pamięciowa, dlatego stwierdzenie, że jest liniowa, byłoby ryzykowne. Aczkolwiek program pobiera mniej pamięci dla tych danych niż przeglądarka Firefox dla kilku otwartych kart, dlatego można założyć, że nie jest to jedna z gorszych złożoności pamięciowych.

3.11.2 Użycie procesora

Na rysunku 3.9 i 3.10 pokazano okno programu dotTrace. Program ten pozwala na analizę czasu pracy procesora, ale najbardziej pomocnym elementem jest sprawdzenie ile % całości używają konkretne metody.



Rysunek 3.9: Okno programu dotTrace - lewy panel - użycie procesora dla operatora AEX.



Rysunek 3.10: Okno programu dotTrace - prawy panel - użycie procesora dla operatora AEX.

Porównanie użycia procesora przez metody dla operatora AEX pokazano w tabeli 3.2. Pominęto metody systemowe czy te pochodzące z frameworków, gdyż nad tym programista nie ma kontroli. W większości pominęto również metody które zajmują mniej niż 0.1% czasu pracy procesora.

Metoda	Użycie procesora
CheckFeasibility	2,8%
DoMutation	2,6%
DoFitness	0,7%
GetCrowdingDistance	0,1%
DoCrossover	0,1%
StartGeneticAlgorithm	0,1%

Tabela 3.2: Porównanie metod pod wpływem użycia procesora dla operatora AEX.

Metoda CheckFeasibility jest elementem operatora AEX która sprawdza, czy element genotypu jest możliwy do przeniesienia do potomka. Opiera się w dużej mierze na operatorach LINQ udostępnionych przez .NET, które mogą prowadzić do sporych narzutów czasowych, więc najprawdopodobniej to jest problemem.

Metoda DoMutation jak sama nazwa mówi odpowiada za mutację. Tutaj problemem może być losowe sortowanie elementów w celu wyboru elementów do podmiany. Gwarantuje to wybór unikatowych elementów bez powtórzeń, ale może być kosztowne. Można zastanowić się nad metodą wydajniejszą, która daje takie same rezultaty.

Metoda DoFitness odpowiada za wyznaczanie wartości kryteriów dla wszystkich osobników. Metoda GetCrowdingDistance wyznacza odległość zagęszczenia dla każdego osobnika.

Metoda StartGeneticAlgorithm zawiera główną pętlę algorytmu genetycznego. Jak widać głównym obciążeniem są operatory, a nie pętla sama w sobie. Czas pracy procesora dla reszty metod wydaje się być w granicach akceptacji, wypada on poniżej 0,1%.

Dla operatora ERX rezultaty przedstawiono w tabeli 3.3. Na pierwszy rzut oka operator ERX bardziej obciąża procesor niż AEX (1.2% a 0.1%), ale AEX ma wyżej wspomnianą mało optymalną metodę CheckFeasibility. Jeżeli to zostanie wzięte pod uwagę, to ERX jest o wiele mniej obciążający dla procesora. Nawet uwzględniając metodę ConstructAdjacencyMatrix, która odpowiada za tworzenie macierzy sąsiedztwa w operatorze ERX.

Rezultaty dla operatora PMX przedstawiono w tabeli 3.4. PMX okazuje się bardzo wydajny. Metoda GetValueFromMap która odpowiada za odczyt z mapy zabiera tak mało czasu pracy procesora, że dotTrace nie pokazuje nawet dokładnej

Metoda	Użycie procesora
DoMutation	3,0%
DoCrossover	1,2%
DoFitness	0,8%
GetCrowdingDistance	0,1%
StartGeneticAlgorithm	0,1%
ConstructAdjacencyMatrix	0,05%

Tabela 3.3: Porównanie metod pod wpływem użycia procesora dla operatora ERX.

Metoda	Użycie procesora
DoMutation	4,0%
DoFitness	1,2%
GetCrowdingDistance	0,2%
StartGeneticAlgorithm	0,08%
DoCrossover	0,04%
GetValueFromMap	<0,01%

Tabela 3.4: Porównanie metod pod wpływem użycia procesora dla operatora PMX.

wartości. To z kolei prowadzi do pewnego przesunięcia skali, przez co mutacja wypada tutaj odrobinę gorzej niż w poprzednich porównaniach.

Podsumowując, wyniki są stosunkowo zadowalające. Aczkolwiek jest również co poprawić. Głównymi kandydatami są metody CheckFeasibility oraz DoMutation. Dla samej pracy i pomiarów efektywności nie ma to dużego znaczenia, ale jeżeli miałyby to być stosowane w praktyce, to wskazana byłaby optymalizacja kodu.

4 Badanie wpływu elementów algorytmu na wynik

W tym rozdziale zostanie dokonana analiza wpływu parametrów algorytmu na wynik. Rezultaty nie będą wizualizowane, analizowane będą tylko wynikowe wartości. Badania przeprowadzono w sposób częściowo heurystyczny, tzn. nie badano takich wartości, gdzie istnieje duża pewność, że wyniki będą gorsze.

Badanie przeprowadzono w następujący sposób - dla konkretnych parametrów i danych wejściowych rejestrowane jest 5 przebiegów. Dla każdego przebiegu wyznaczane są wartości:

- Iteracja (Iter) - iteracja znalezienia osobnika najlepiej spełniającego wszystkie trzy kryteria,
- Średnia poprawa (Avg Impr) - wartość reprezentująca o ile % zmieniły się wszystkie trzy kryteria dla osobnika najlepiej spełniającego wszystkie trzy kryteria - jest to średnia wartość następujących trzech wartości,
- Poprawa popularności (Pop Impr) - o ile % spadło pierwsze kryterium oceny,
- Poprawa wagi (Weight Impr) - o ile % spadło drugie kryterium oceny,
- Poprawa dystansów (Dist Impr) - o ile % spadło trzecie kryterium oceny,
- Średnia popularności (Avg Pop) - średnia wartość pierwszego kryterium w ostatniej iteracji dla całej populacji,
- Średnia wagi (Avg Weight) - średnia wartość drugiego kryterium w ostatniej iteracji dla całej populacji,
- Średnia dystansów (Avg Dist) - średnia wartość trzeciego kryterium w ostatniej iteracji dla całej populacji.

Następnie w tabeli rejestrowane są średnie wartości dla wszystkich przebiegów, oraz dodatkowo notowane jest odchylenie standardowe tych średnich. Zastosowano skróty nazw wartości aby zminimalizować rozmiar tabel.

Przedstawione wartości to nie wszystko co można przeanalizować, ale powinny być wystarczające do obiektywnej oceny efektywności. Wyczerpująca analiza byłaby bardzo czasochłonna, a rezultaty nie mieściłyby się w pojedynczych tabelach.

Należy również wspomnieć o sensie wartości poprawy w badaniach. Żeby wartości te były miarodajne, użyto stałego seedu gwarantującego, że punkt startowy populacji jest zawsze identyczny. Po pierwsze gwarantuje to obiektywność badań - za każdym razem algorytm startuje z tego samego punktu. Po drugie pozwala to na czytelne pomiary w % - im większy procent, tym lepsza poprawa. Gdyby

punkt startowy był losowy, taki sam procent może oznaczać zupełnie inne wartości, i badania byłyby całkowicie niemiernodajne. Dodatkowo trzeba by analizować realne wartości pojedynczych kryteriów.

4.1 Badanie operatora AEX

Celem tego podrozdziału jest znalezienie jak najlepszych parametrów wejściowych dla operatora AEX, oraz zbadanie, jak zmienia się rezultat działania algorytmu genetycznego.

Badanie przeprowadzono na magazynie o pojemności 132 towarów. Wstępne parametry wejściowe:

- Fitness - równoległy (współbieżny),
- Odległość zagęszczenia - równoległa (współbieżna),
- Rozmiar populacji - 80,
- Seed populacji - stały, 5585,
- Liczba iteracji - 200,
- Liczba rodziców - 2,
- Prawdopodobieństwo mutacji - 60%,
- Liczba mutowanych osobników o niskiej wartości fitness - 60%,
- Liczba podmian elementów genotypu w mutacji - od 20 do 40.

Liczbę iteracji również należy dobrać eksperymentalnie, nie jest z góry znana optymalna liczba. W wielu przypadkach około 100 jest wystarczające, dlatego założono liczbę 200. Powinno to zagwarantować, że algorytm nie zostanie zakończony przedwcześnie, i wynik powinien zostać mocno poprawiony przed zakończeniem.

W niektórych przypadkach użyto pomiarów z badań w podrozdziałach poprzedzających, jeżeli wykonano już pomiary dla takiej samej konfiguracji, aby przyspieszyć proces badawczy.

4.1.1 Rozmiar populacji

Pierwszym krokiem jest znalezienie optymalnej ilości osobników w populacji. Wartości kryteriów spadają do mniej więcej 150 iteracji, po czym sytuacja się stabilizuje. Konkretnie rezultaty przedstawiono w tabeli 4.1. Na początku można zauważyć, że wartości średniej poprawy kryterium 1 i 2 są bardzo niskie. Nie

	Rozmiar populacji				
	80	90	100	110	120
Iter	132	150	153	80	137
Iter StdDev	32	42	29	50	54
Avg Impr	9,844%	10,443%	10,900%	10,944%	12,762%
Pop Impr	0,215%	0,055%	-1,553%	-1,700%	0,628%
Weight Impr	-0,363%	-0,496%	2,123%	1,118%	1,290%
Dist Impr	29,678%	31,768%	32,131%	33,415%	36,368%
Avg Impr StdDev	1,465%	1,262%	1,805%	0,713%	0,620%
Pop Impr StdDev	1,690%	1,275%	3,310%	1,511%	1,489%
Weight Impr StdDev	2,129%	1,307%	1,353%	2,191%	2,115%
Dist Impr StdDev	3,580%	3,220%	8,045%	2,762%	4,328%
Avg Pop	6440950,36	6454958,20	6446368,07	6464804,06	6433381,35
Avg Weight	6534638,03	6534820,16	6543752,17	6532732,68	6531480,69
Avg Dist	621909,64	625490,13	639968,40	617348,56	605575,80
Avg Pop StdDev	6915,17	26476,35	7830,78	13848,07	30448,45
Avg Weight StdDev	9818,70	18039,32	13678,77	8678,34	31183,54
Avg Dist StdDev	8586,11	10817,94	11586,47	6898,15	21122,30

Tabela 4.1: Wpływ rozmiaru populacji dla AEX.

oznacza to, że algorytm nie działa - w zbiorze rozwiązań Pareto optymalnych są rozwiązania które całkiem sensownie poprawiają wartości tych kryteriów, ale pogarszają inne kryteria na tyle, że żaden z nich nie został uznany za rozwiązanie które najlepiej spełnia wszystkie trzy kryteria. Wniosek jest jeden - operator AEX wstępnie wydaje się być mało efektywny w wyznaczaniu rozwiązań które dobrze spełniają wszystkie kryteria, ale na ten moment jest za wcześnie żeby wyciągać konkretne wnioski.

Patrząc na wyniki w tabeli, rozmiar populacji poniżej 100 wydaje się być zbyt mały. Współczynniki poprawy nie są wystarczająco dobre. Dla 80 odchylenie standardowe średniej kryteriów całej populacji jest małe, co może oznaczać coś w rodzaju przedwczesnej konwergencji. Dla 90 jest zbyt duże. Dla 100 rezultaty wydają się być sensowne, a odchylenie standardowe wartości kryteriów dla populacji wygląda na zawierające się w granicach akceptowalności. Dla 110 sytuacja również wygląda dobrze, jeśli nawet nie lepiej pod pewnymi względami. 120 daje jeszcze lepsze wyniki, ale odchylenie standardowe zaczyna rosnąć, co sugeruje bardziej chaotyczne zachowanie i problemy z konwergencją.

Z tego można wywnioskować, że optymalnym rozmiarem populacji jest od 100 do 110 osobników. Dla dalszych testów wybrano opcję 100.

4.1.2 Prawdopodobieństwo mutacji

Wyniki przedstawiono w tabeli 4.2. Okazuje się, że prawdopodobieństwo 70% jest lepsze niż 60%, a 80% to wartość zbyt duża, która pogarsza wynik. Co prawda mutacja o prawdopodobieństwie 70% zwiększa odchylenie standardowe wartości kryteriów całej populacji, ale biorąc pod uwagę wyraźne polepszenie wyniku, nie ma co się obawiać problemów ze zbieżnością. Dla 70% wzrasta losowość iteracji znalezionej wartości, ale nie tak mocno jak dla 80%. Prawdopodobieństwo mutacji ustalono jako 70%.

4.1.3 Ilość mutowanych osobników

Rezultaty przedstawiono w tabeli 4.3. Wartość 60% daje dosyć sensowne wyniki. 70% daje wyniki gorsze, ale za to 80% daje wyniki jeszcze lepsze. W tym momencie ciężko stwierdzić czy faktycznie 70% jest złą opcją, czy po prostu losowość była w tym momencie niekorzystna. Dla 70% i 80% również częściowo spada odchylenie standardowe średniej kryteriów dla wszystkich osobników, co może sugerować większą zbieżność.

Dla 70% i 80% zachowanie zaczyna być bardziej chaotyczne, co jest głównie sugerowane przez moment znalezienia rozwiązania. Można uznać 60% za wartość bezpieczną i stabilną, a wartości większe jeżeli użytkownik chce zaryzykować

	Prawdopodobieństwo mutacji		
	60%	70%	80%
Iter	153	139	84
Iter StdDev	29	48	65
Avg Impr	10,900%	11,215%	10,445%
Pop Impr	-1,553%	0,549%	2,089%
Weight Impr	2,123%	0,644%	-2,022%
Dist Impr	32,131%	32,451%	31,267%
Avg Impr StdDev	1,805%	1,053%	0,944%
Pop Impr StdDev	3,310%	0,854%	1,358%
Weight Impr StdDev	1,353%	0,864%	2,133%
Dist Impr StdDev	8,045%	3,366%	3,827%
Avg Pop	6446368,07	6454447,72	6464853,61
Avg Weight	6543752,17	6548309,65	6553390,46
Avg Dist	639968,40	625281,19	619609,51
Avg Pop StdDev	7830,78	31805,68	20618,57
Avg Weight StdDev	13678,77	11252,08	7269,05
Avg Dist StdDev	11586,47	18174,31	14300,27

Tabela 4.2: Wpływ prawdopodobieństwa mutacji dla AEX.

	Ilość mutowanych osobników		
	60%	70%	80%
Iter	139	69	85
Iter StdDev	48	43	43
Avg Impr	11,215%	10,653%	11,327%
Pop Impr	0,549%	-1,731%	-0,234%
Weight Impr	0,644%	1,243%	-1,020%
Dist Impr	32,451%	32,446%	35,236%
Avg Impr StdDev	1,053%	1,744%	1,427%
Pop Impr StdDev	0,854%	2,037%	3,185%
Weight Impr StdDev	0,864%	2,014%	0,453%
Dist Impr StdDev	3,366%	5,190%	3,190%
Avg Pop	6454447,72	6466535,72	6505515,23
Avg Weight	6548309,65	6563600,61	6590287,78
Avg Dist	625281,19	647983,00	656454,83
Avg Pop StdDev	31805,68	17595,26	16291,66
Avg Weight StdDev	11252,08	17705,57	17726,75
Avg Dist StdDev	18174,31	19533,81	9771,06

Tabela 4.3: Wpływ ilości mutowanych osobników dla AEX.

lepszy wynik, zgadzając się na bardziej losowe zachowanie, i potencjalne straty jakości. Do dalszych testów wybrano opcję 60%.

4.1.4 Liczba podmian elementów genotypu

Wyniki przedstawiono w tabeli 4.4. Przedział 40-60 jest wyraźnie lepszy

	Liczba podmian elementów genotypu		
	20-40	40-60	60-80
Iter	139	114	105
Iter StdDev	48	59	49
Avg Impr	11,215%	12,608%	11,246%
Pop Impr	0,549%	0,659%	-1,181%
Weight Impr	0,644%	0,916%	-0,261%
Dist Impr	32,451%	36,247%	35,179%
Avg Impr StdDev	1,053%	0,681%	1,249%
Pop Impr StdDev	0,854%	2,136%	1,355%
Weight Impr StdDev	0,864%	1,848%	1,071%
Dist Impr StdDev	3,366%	4,223%	3,326%
Avg Pop	6454447,72	6443821,88	6469842,12
Avg Weight	6548309,65	6550970,68	6554687,06
Avg Dist	625281,19	623200,13	632451,65
Avg Pop StdDev	31805,68	15487,06	16735,29
Avg Weight StdDev	11252,08	13427,03	26347,02
Avg Dist StdDev	18174,31	27065,53	13105,45

Tabela 4.4: Wpływ liczby podmian elementów genotypu dla AEX.

od 20-40 - średnia poprawa jest znacznie lepsza pod każdym względem. Przedział 60-80 wydaje się być lepszy niż 20-40, ale rezultaty zaczynają być już mało spójne. Ocena zbieżności jest ciężka, gdyż rezultaty są trochę chaotyczne pod tym względem, ale rezultaty dla 40-60 nie odstają od normy.

Warto również dodać, że dla silniejszej mutacji zaobserwowano przesunięcie wykresu, tzn. gdy dla poprzednich przypadków fitness spadał mniej więcej do iteracji 150, z bardzo niewielkimi poprawkami w późniejszych iteracjach, teraz

zaobserwowano znaczną poprawę nawet do 190 iteracji. Silniejsza mutacja rzeczywiście chroni przed przedwczesną konwergencją, ale przedział 60-80 wydaje się być zbyt silny.

Podsumowując, liczbę podmian elementów genotypu ustalono jako 40-60. Jako, że długość genotypu wynosi 132, to zakres ten w procentach powinien wynosić około 30% do 45% długości genotypu.

4.1.5 Liczba rodziców

AEX akceptuje dowolną liczbę rodziców od 2 wzwyż. Przeprowadzono testy dla 2, 4, oraz 8 rodziców. Wyniki przedstawiono w tabeli 4.5.

	Liczba rodziców		
	2	4	8
Iter	114	122	108
Iter StdDev	59	73	54
Avg Impr	12,608%	10,987%	9,738%
Pop Impr	0,659%	-1,655%	-0,217%
Weight Impr	0,916%	-1,298%	0,195%
Dist Impr	36,247%	35,914%	29,236%
Avg Impr StdDev	0,681%	0,583%	1,745%
Pop Impr StdDev	2,136%	2,975%	1,192%
Weight Impr StdDev	1,848%	1,717%	1,417%
Dist Impr StdDev	4,223%	2,401%	6,351%
Avg Pop	6443821,88	6477084,64	6463391,16
Avg Weight	6550970,68	6553719,87	6570801,78
Avg Dist	623200,13	629172,33	620008,75
Avg Pop StdDev	15487,06	13415,18	10473,16
Avg Weight StdDev	13427,03	8012,04	15687,84
Avg Dist StdDev	27065,53	6559,04	13293,53

Tabela 4.5: Wpływ liczby rodziców dla AEX.

Jak widać większa liczba rodziców ma negatywny wpływ na działanie algorytmu, dlatego pominięto badanie większej liczby rodziców. Wraz ze wzrostem

liczby rodziców poprawa maleje. Można podejrzewać, że ma to negatywny wpływ na różnorodność populacji, co utrudnia znalezienie rozwiązania. Używanie operatora AEX z dwoma rodzicami jest najlepszym rozwiązaniem.

4.1.6 Badanie operatora AEX dla większego magazynu

Badania zakończono ustalając następujące parametry:

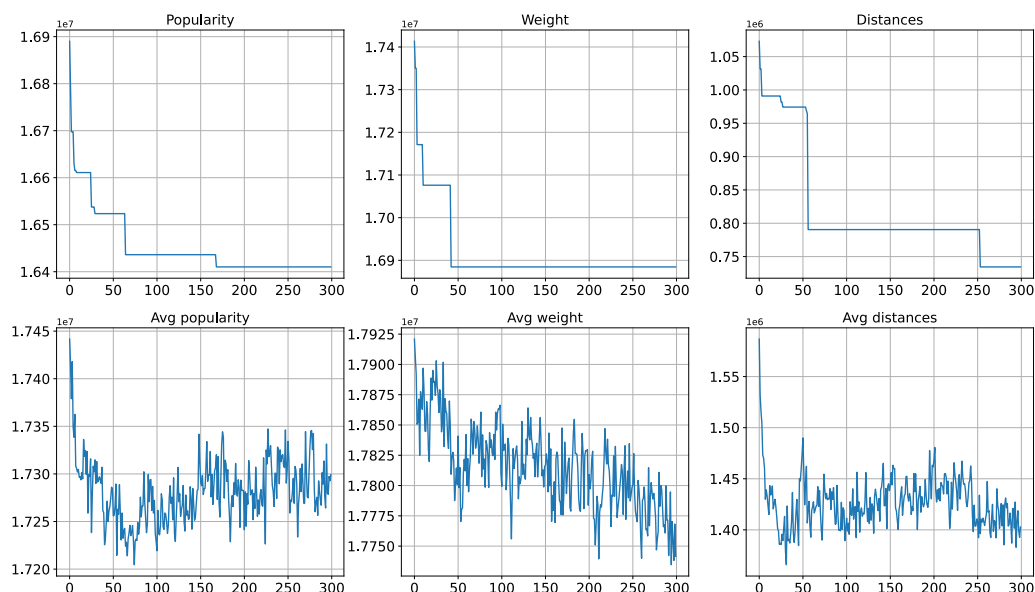
- Rozmiar populacji - 100,
- Liczba rodziców - 2,
- Prawdopodobieństwo mutacji - 70%,
- Liczba mutowanych osobników - 60%,
- Liczba podmian w mutacji - 40 - 60 (30% - 45% długości genotypu).

Dla takich samych danych przeprowadzono dodatkowy test - analiza jednego przebiegu dla większego magazynu który mieści 240 towarów. Liczba podmian w mutacji konkretnie wynosi od 72 do 108, ze względu na dłuższą długość genotypu dla większego magazynu.

Teraz analizie poddane zostaną częściowo inne wartości niż poprzednio, aby mieć pełny obraz działania algorytmu. Użyto innego seedu - 2248. Później zostanie wykonane porównanie operatorów, i potrzebny jest stały punkt startowy. Ilość iteracji zwiększono do 300. Uzyskano następujące rezultaty:

- Średnia poprawa - 13,393%,
- Poprawa popularności - -2,133%,
- Poprawa wagi - 1,941%,
- Poprawa dystansów - 40,372%,
- Najlepsza poprawa popularności - 6,137%,
- Najlepsza poprawa wagi - 6,449%,
- Najlepsza poprawa dystansów - 40,372%,

Przebieg algorytmu przedstawiono na wykresach 4.1. Każde z kryteriów jest poprawiane, aczkolwiek poprawa nie jest regularna, istnieją długie okresy gdzie wartości są stałe, największe spadki występują na początku. Średnie spadają, aczkolwiek spadek nie jest mocny, a w przypadku kryterium pierwszego można zaobserwować nawet wzrosty.



Rysunek 4.1: Przebieg algorytmu dla dużego magazynu dla operatora AEX.

4.1.7 Podsumowanie badań operatora AEX

Ogólnie rzecz biorąc rezultaty mieszczą się w granicach akceptacji, dla każdego kryterium następowała całkiem sensowna poprawa. Pełnej puli rozwiązań Pareto optymalnych nie zaprezentowano, ale dla kryterium popularności i wag poprawa również była korzystna, nawet do około 10%. Jedyną obiektywną wadą są problemy ze znalezieniem rozwiązania które dobrze spełnia wszystkie kryteria. Niestety tam gdzie kryteria popularności i wagi były poprawione, występowały znaczne spadki kryterium dystansów.

Ze względu na testowanie algorytmu tylko na jednym zbiorze danych (na drugim magazynie wykonano tylko jeden test) ocena nie jest w pełni obiektywna, ale jest to coś co trzeba mieć na uwadze. Operator AEX można uznać za działający poprawnie, ale gdy ważne jest znalezienie rozwiązania które dobrze spełnia wszystkie trzy kryteria jego efektywność nie wydaje się być bardzo wysoka.

4.2 Badanie operatora ERX

Zmiana operatora nie powinna bardzo wpływać na zachowanie algorytmu jeśli chodzi o parametry wejściowe. Aczkolwiek dla pewności przeprowadzono badania sprawdzając wartości mniejsze i większe dla parametrów uzyskanych dla operatora AEX.

4.2.1 Rozmiar populacji

Rezultaty badania wpływu rozmiaru populacji przedstawiono w tabeli 4.6. 100 osobników w populacji daje niepodważalnie lepsze rezultaty niż 90. 110 jest

	Rozmiar populacji		
	90	100	110
Iter	151	128	136
Iter StdDev	58	43	69
Avg Impr	11,459%	13,525%	13,012%
Pop Impr	-0,367%	-0,496%	1,158%
Weight Impr	1,006%	0,816%	-0,047%
Dist Impr	33,737%	40,254%	37,926%
Avg Impr StdDev	1,718%	1,649%	2,819%
Pop Impr StdDev	2,318%	1,209%	3,609%
Weight Impr StdDev	2,088%	2,081%	1,429%
Dist Impr StdDev	5,817%	3,860%	8,460%
Avg Pop	6473750,71	6468214,95	6463819,83
Avg Weight	6559029,12	6558444,93	6547332,34
Avg Dist	623118,82	615957,82	600356,01
Avg Pop StdDev	21320,12	7934,59	10467,42
Avg Weight StdDev	15815,98	10550,92	20704,46
Avg Dist StdDev	9407,68	15461,57	42164,90

Tabela 4.6: Wpływ rozmiaru populacji dla ERX.

również lepsze od 90, ale w tym momencie zachowanie zaczyna być bardziej chaotyczne, i potencjalne zyski nie są warte dużych i częstych strat. Dlatego 100 osobników wydaje się być najlepszym wyborem.

4.2.2 Prawdopodobieństwo mutacji

Rezultaty badania wpływu prawdopodobieństwa mutacji przedstawiono w tabeli 4.7. Okazuje się, że dla ERX bardziej opłacalne jest zmniejszenie prawdopodobieństwa mutacji. Wyniki są niepodważalnie lepsze, bez dużego zwiększenia

	Prawdopodobieństwo mutacji		
	60%	70%	80%
Iter	148	128	101
Iter StdDev	44	43	57
Avg Impr	14,783%	13,525%	12,347%
Pop Impr	2,145%	-0,496%	1,277%
Weight Impr	0,677%	0,816%	1,679%
Dist Impr	41,527%	40,254%	34,087%
Avg Impr StdDev	2,402%	1,649%	1,030%
Pop Impr StdDev	1,278%	1,209%	2,853%
Weight Impr StdDev	1,974%	2,081%	1,498%
Dist Impr StdDev	6,857%	3,860%	4,085%
Avg Pop	6384621,46	6468214,95	6465493,10
Avg Weight	6515414,63	6558444,93	6546704,58
Avg Dist	555013,70	615957,82	641715,72
Avg Pop StdDev	90046,37	7934,59	24500,40
Avg Weight StdDev	35743,77	10550,92	12957,40
Avg Dist StdDev	91802,00	15461,57	10507,99

Tabela 4.7: Wpływ prawdopodobieństwa mutacji dla ERX.

odchylenia standardowego. Odchylenie standardowe całkowitej populacji jest większe, ale to niekoniecznie jest wadą, widocznie dochodzi do wzrostu zróżnicowania, co prowadzi do lepszych wyników. Zwiększenie siły mutacji prowadzi do trochę gorszych rezultatów.

4.2.3 Ilość mutowanych osobników

Rezultaty badania przedstawiono w tabeli 4.8. Mutowanie 60% osobni-

	Ilość mutowanych osobników		
	50%	60%	70%
Iter	167	148	116
Iter StdDev	46	44	36
Avg Impr	14,733%	14,783%	11,308%
Pop Impr	0,882%	2,145%	-1,207%
Weight Impr	-0,293%	0,677%	1,165%
Dist Impr	43,611%	41,527%	33,966%
Avg Impr StdDev	4,117%	2,402%	1,534%
Pop Impr StdDev	2,260%	1,278%	0,727%
Weight Impr StdDev	3,095%	1,974%	1,503%
Dist Impr StdDev	9,584%	6,857%	4,589%
Avg Pop	6410632,74	6384621,46	6482852,58
Avg Weight	6528082,03	6515414,63	6573579,56
Avg Dist	521097,16	555013,70	645362,86
Avg Pop StdDev	57601,48	90046,37	4401,83
Avg Weight StdDev	40501,62	35743,77	7778,20
Avg Dist StdDev	95254,99	91802,00	9170,04

Tabela 4.8: Wpływ ilości mutowanych osobników dla ERX.

ków daje rezultaty najbardziej spójne. 70% nie daje już dobrych wyników. 50% zachowuje się trochę chaotycznie, niekiedy dając lepsze wyniki, co prowadzi do większego odchylenia standardowego. 60% zostaje uznane za najlepsze, ale 50% wcale nie jest bardzo złe, i można próbować również używać tej opcji.

4.2.4 Liczba podmian elementów genotypu

Rezultaty badań przedstawiono w tabeli 4.9. Zakres 40-60 daje rezultaty

	Liczba podmian elementów genotypu		
	20-40	40-60	60-80
Iter	131	148	130
Iter StdDev	39	44	54
Avg Impr	11,289%	14,783%	12,908%
Pop Impr	-0,331%	2,145%	0,284%
Weight Impr	0,351%	0,677%	0,485%
Dist Impr	33,846%	41,527%	37,954%
Avg Impr StdDev	1,858%	2,402%	1,380%
Pop Impr StdDev	1,658%	1,278%	2,618%
Weight Impr StdDev	2,295%	1,974%	2,145%
Dist Impr StdDev	8,859%	6,857%	1,769%
Avg Pop	6452598,14	6384621,46	6456261,43
Avg Weight	6551650,99	6515414,63	6567437,37
Avg Dist	621477,81	555013,70	621160,38
Avg Pop StdDev	7803,02	90046,37	13716,73
Avg Weight StdDev	14043,71	35743,77	15549,52
Avg Dist StdDev	9371,76	91802,00	15209,12

Tabela 4.9: Wpływ liczby podmian elementów genotypu dla ERX.

najlepsze. 20-40 to wyraźnie za mało, a 60-80 to już zbyt dużo - wynik zaczyna się pogarszać. 40-60 to w takim razie odpowiedni wybór. Co prawda ma stosunkowo duże odchylenie standardowe dla całej populacji, ale jak widać dzięki temu jakość wyniku jest lepsza.

4.2.5 Liczba rodziców

Rezultaty badania przedstawiono w tabeli 4.10. Jakość wyniku zaczyna drastycznie spadać od 8 rodziców, dlatego nie kontynuowano badań dla większych wartości. Operator ERX dla 2 rodziców daje rezultaty trochę lepsze niż dla 4,

	Liczba rodziców		
	2	4	8
Iter	148	175	166
Iter StdDev	44	23	30
Avg Impr	14,783%	14,555%	11,098%
Pop Impr	2,145%	2,661%	0,509%
Weight Impr	0,677%	0,454%	-0,275%
Dist Impr	41,527%	40,551%	33,059%
Avg Impr StdDev	2,402%	1,836%	1,845%
Pop Impr StdDev	1,278%	1,898%	1,264%
Weight Impr StdDev	1,974%	1,101%	0,955%
Dist Impr StdDev	6,857%	5,557%	5,146%
Avg Pop	6384621,46	6380777,81	6434442,43
Avg Weight	6515414,63	6534672,72	6543714,97
Avg Dist	555013,70	532663,71	595170,58
Avg Pop StdDev	90046,37	61844,83	40736,68
Avg Weight StdDev	35743,77	18177,77	28166,36
Avg Dist StdDev	91802,00	61112,58	34858,64

Tabela 4.10: Wpływ liczby rodziców dla ERX.

ale kosztem większej losowości. Dla 4 rezultaty są bardziej spójne. Wybrano 4 rodziców jako opcję najlepszą.

4.2.6 Badanie operatora ERX dla większego magazynu

Badania zakończono ustalając następujące parametry:

- Rozmiar populacji - 100,
- Liczba rodziców - 4,
- Prawdopodobieństwo mutacji - 60%,
- Liczba mutowanych osobników - 60%,
- Liczba podmian w mutacji - 40 - 60 (30% - 45% długości genotypu).

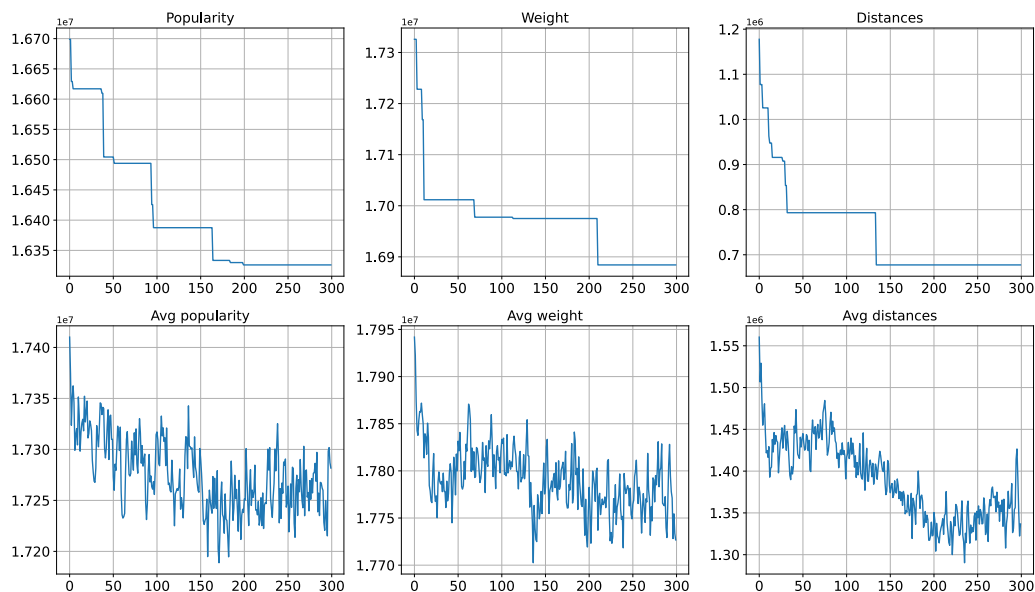
Test dla większego magazynu wykonano analogicznie jak dla operatora AEX. Ponownie liczba podmian w mutacji wynosi od 72 do 108, a seed to 2248. Uzyskano następujące rezultaty:

- Średnia poprawa - 15,037%,
- Poprawa popularności - -0,066%,
- Poprawa wagi - 0,179%,
- Poprawa dystansów - 44,999%,
- Najlepsza poprawa popularności - 6,619%,
- Najlepsza poprawa wagi - 6,451%,
- Najlepsza poprawa dystansów - 44,999%,

Przebieg algorytmu przedstawiono na wykresach 4.2. Zachowanie jest podobne jak w przypadku AEX, aczkolwiek spadki wartości są bardziej regularne, a średnie mają mocniejszą tendencję spadkową.

4.2.7 Podsumowanie badań operatora ERX

Finałowe rezultaty nie różnią się znacznie od operatora AEX. Również istnieją problemy ze znalezieniem rozwiązania które dobrze spełnia wszystkie trzy kryteria. Aczkolwiek operator daje minimalnie lepsze rezultaty. Częściowo deklasuje to operator AEX, ale ze względu na subtelne różnice ciężko zdecydować, czy faktycznie ERX jest obiektywnie lepszy. Być może istnieją problemy gdzie AEX będzie dawał lepsze rozwiązania. Na pewno można wywnioskować, że ERX nie jest nieefektywny mimo bycia stworzonym do rozwiązywania problemu komiwojażera.



Rysunek 4.2: Przebieg algorytmu dla dużego magazynu dla operatora ERX.

4.3 Badanie operatora PMX

Badanie operatora PMX przeprowadzono podobnie jak ERX - wystartowano z parametrami wyznaczonymi dla AEX, i sprawdzano czy pewne parametry trzeba zmniejszyć czy zwiększyć.

4.3.1 Rozmiar populacji

Rezultaty badania przedstawiono w tabeli 4.11. Różnice są dosyć subtelne, ale zauważalne. 110 daje odrobinę gorsze wyniki od reszty, a 100 mimo minimalnie gorszej średniej poprawy od 90 ma mniejsze wartości odchyleń standardowych, co sugeruje większą zbieżność.

Rozwiązania najlepiej spełniające wszystkie trzy kryteria są znajdowane stosunkowo późno. Wynika to z dosyć wysokiej efektywności operatora PMX względem poprzednich operatorów. Wynik mocno poprawia się do około 100 iteracji, po czym poprawa zaczyna spowalniać, ale wciąż jest mocno widoczna. W praktyce można spróbować zwiększyć ilość iteracji, ale żeby porównanie było sprawiedliwe limit w dalszym ciągu ustawiony jest na 200. Podsumowując, 100 osobników w dalszym ciągu wydaje się być dobrym wyborem, tak jak dla poprzednich operatorów.

	Rozmiar populacji		
	90	100	110
Iter	183	182	183
Iter StdDev	15	16	10
Avg Impr	24,128%	24,112%	24,092%
Pop Impr	4,457%	3,787%	2,553%
Weight Impr	4,474%	4,822%	4,679%
Dist Impr	63,454%	63,728%	65,044%
Avg Impr StdDev	1,751%	1,320%	1,502%
Pop Impr StdDev	4,007%	3,199%	4,210%
Weight Impr StdDev	2,215%	1,519%	1,370%
Dist Impr StdDev	1,639%	2,607%	1,048%
Avg Pop	6125648,87	6158763,20	6156589,09
Avg Weight	6179290,33	6184079,01	6185826,37
Avg Dist	516364,75	495315,07	508100,24
Avg Pop StdDev	69893,67	64668,22	55088,07
Avg Weight StdDev	44561,94	21792,63	47331,92
Avg Dist StdDev	27112,15	30630,00	43149,97

Tabela 4.11: Wpływ rozmiaru populacji dla PMX.

4.3.2 Prawdopodobieństwo mutacji

Rezultaty badania przedstawiono w tabeli 4.12. Patrząc się tylko na średnią

	Prawdopodobieństwo mutacji		
	60%	70%	80%
Iter	170	182	183
Iter StdDev	19	16	9
Avg Impr	23,777%	24,112%	23,929%
Pop Impr	3,231%	3,787%	4,883%
Weight Impr	4,029%	4,822%	4,319%
Dist Impr	64,071%	63,728%	62,585%
Avg Impr StdDev	0,841%	1,320%	1,098%
Pop Impr StdDev	1,058%	3,199%	1,103%
Weight Impr StdDev	1,031%	1,519%	1,743%
Dist Impr StdDev	0,747%	2,607%	1,913%
Avg Pop	6118742,13	6158763,20	6182389,81
Avg Weight	6133774,19	6184079,01	6267184,26
Avg Dist	483040,94	495315,07	527767,69
Avg Pop StdDev	45761,78	64668,22	41988,27
Avg Weight StdDev	64385,90	21792,63	52832,64
Avg Dist StdDev	31714,80	30630,00	14933,88

Tabela 4.12: Wpływ prawdopodobieństwa mutacji dla PMX.

poprawę prawdopodobieństwo mutacji 70% daje najlepsze wyniki. Ogólnie różnice są bardzo subtelne, i prawdopodobnie każda z tych konfiguracji będzie dawać dobre rezultaty. Aczkolwiek odrobinę gorsze wyniki mutacji 80% sugerują, że jest to już zbyt wysoki poziom losowości. Mutacja 60% zwraca trochę gorsze wyniki niż 70%, ma częściowo większe odchylenie standardowe całej populacji, ale za to mniejsze odchylenie standardowe wyników. Wybór mutacji 60% nie powinien być zły, ale żeby mieć pewność, że faktycznie różnorodność osobników jest wystarczająca, wybrano prawdopodobieństwo mutacji 70%.

4.3.3 Ilość mutowanych osobników

Rezultaty badania przedstawiono w tabeli 4.13. Ciężko jednoznacznie

	Ilość mutowanych osobników		
	50%	60%	70%
Iter	194	182	190
Iter StdDev	5	16	7
Avg Impr	24,222%	24,112%	23,796%
Pop Impr	8,789%	3,787%	5,555%
Weight Impr	2,421%	4,822%	1,854%
Dist Impr	61,457%	63,728%	63,978%
Avg Impr StdDev	1,386%	1,320%	0,851%
Pop Impr StdDev	1,732%	3,199%	2,242%
Weight Impr StdDev	2,327%	1,519%	1,978%
Dist Impr StdDev	4,698%	2,607%	2,506%
Avg Pop	6095129,62	6158763,20	6164339,20
Avg Weight	6102749,21	6184079,01	6332787,31
Avg Dist	495695,19	495315,07	534945,84
Avg Pop StdDev	73906,68	64668,22	37548,18
Avg Weight StdDev	41607,65	21792,63	26946,28
Avg Dist StdDev	18326,64	30630,00	34957,88

Tabela 4.13: Wpływ ilości mutowanych osobników dla PMX.

stwierdzić która opcja jest obiektywnie najlepsza, gdyż pod każdym względem niektóre wartości są lepsze, a inne gorsze. 70% daje trochę gorszą poprawę, dlatego można założyć, że jest to wartość zbyt duża. Rezultaty dla 50% i 60% mieszczą się w normie. Dla dalszych badań wybrano opcję 60%, ze względu na częściowo mniejsze odchylenia standardowe wartości kryteriów niż 50%. Dla realnego zastosowania 60% jak i 50% nie powinny dawać złych rezultatów.

4.3.4 Liczba podmian elementów genotypu

Rezultaty badania przedstawiono w tabeli 4.14. Ponownie ciężko jednoznacznie stwierdzić który wariant jest najlepszy. 20-40 daje trochę gorsze rezultaty,

	Liczba podmian elementów genotypu		
	20-40	40-60	60-80
Iter	190	182	188
Iter StdDev	10	16	9
Avg Impr	23,370%	24,112%	24,386%
Pop Impr	4,841%	3,787%	7,633%
Weight Impr	3,633%	4,822%	3,900%
Dist Impr	61,637%	63,728%	61,624%
Avg Impr StdDev	1,237%	1,320%	1,733%
Pop Impr StdDev	4,325%	3,199%	2,096%
Weight Impr StdDev	3,876%	1,519%	2,981%
Dist Impr StdDev	4,170%	2,607%	3,447%
Avg Pop	6159388,74	6158763,20	6084930,39
Avg Weight	6188080,21	6184079,01	6189077,59
Avg Dist	550474,40	495315,07	499749,05
Avg Pop StdDev	16927,74	64668,22	64489,63
Avg Weight StdDev	21000,52	21792,63	63144,60
Avg Dist StdDev	46088,31	30630,00	24260,54

Tabela 4.14: Wpływ liczby podmian elementów genotypu dla PMX.

więc jest to wartość prawdopodobnie zbyt mała. 60-80 potrafi zwrócić odrobinę lepszy wynik niż 40-60, ale kosztem częściowo większego odchylenia standardowego. Dlatego oba przedziały są odpowiednią opcją. Dla dalszych badań bardziej stonowana opcja 40-60 zostaje wybrana jako ta najlepsza. W praktyce wybór 60-80 również powinien być dobry, w szczególności jeżeli użytkownik chce zaryzykować lepsze wyniki kosztem potencjalnych spadków wartości pojedynczych kryteriów.

4.3.5 Badanie operatora PMX dla większego magazynu

Badania zakończono wybierając następujące parametry:

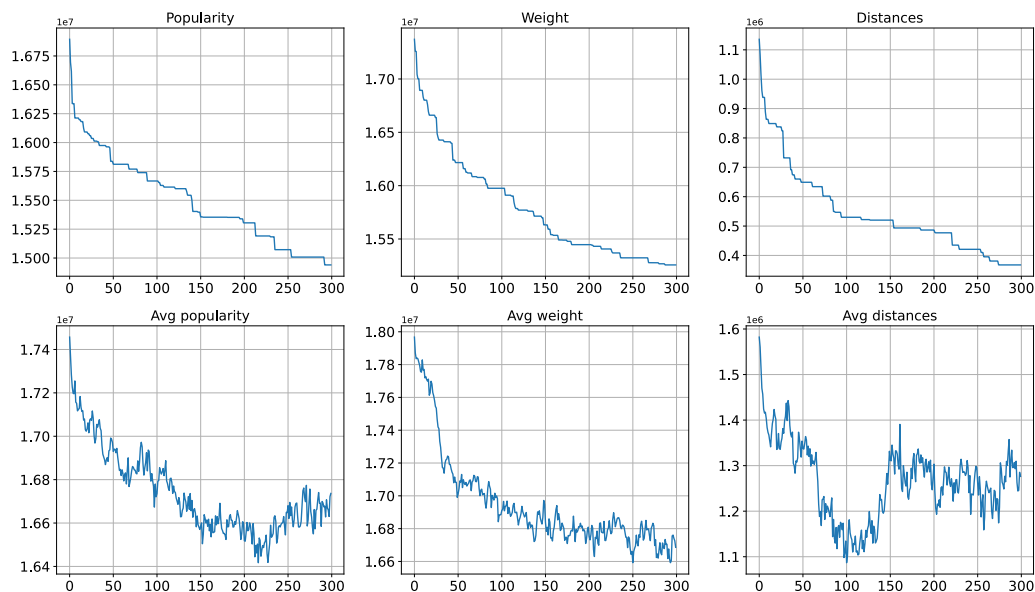
- Rozmiar populacji - 100,
- Prawdopodobieństwo mutacji - 70%,
- Liczba mutowanych osobników - 60%,
- Liczba podmian w mutacji - 40 - 60 (30% - 45% długości genotypu).

Czyli są to parametry identyczne jak dla operatora AEX. Pominięto badanie liczby rodziców, gdyż jak wspomniano wcześniej operator PMX może operować tylko na dwóch rodzicach.

Test dla większego magazynu wykonano analogicznie jak dla poprzednich operatorów, ponownie liczba podmian w mutacji wynosi od 72 do 108, a seed to 2248. Uzyskano następujące rezultaty:

- Średnia poprawa - 26,417%,
- Poprawa popularności - 5,085%,
- Poprawa wagi - 4,018%,
- Poprawa dystansów - 70,149%,
- Najlepsza poprawa popularności - 14,545%,
- Najlepsza poprawa wagi - 15,467%,
- Najlepsza poprawa dystansów - 70,149%,

Przebieg algorytmu przedstawiono na wykresach 4.3. Każde z kryteriów jest ciągle i regularnie poprawiane. Średnie wartości mają tendencję spadkową, aczkolwiek jak widać w przypadku średniej kryterium trzeciego doszło do nagłego wzrostu. Nie ma to jednak aż tak dużego znaczenia gdy ciągle następuje dobra poprawa.



Rysunek 4.3: Przebieg algorytmu dla dużego magazynu dla operatora PMX.

4.3.6 Podsumowanie badań operatora PMX

Operator PMX daje bardzo dobre rezultaty. Potrafi znaleźć wynik który sensownie spełnia wszystkie kryteria, a w puli rozwiązań są wyniki które bardzo dobrze spełniają pojedyncze kryteria. Działa on również bardzo szybko ze względu na niewielką złożoność obliczeniową, oraz zwrot dwóch osobników podczas jednego krzyżowania.

4.4 Porównanie operatorów

Porównanie poprzednio wyznaczonych rezultatów przedstawiono w tabeli 4.15. Użyto takich samych skrótów opisu wierszy jak w poprzednich tabelach, ale dodano wartość najlepszą, w tabeli oznaczoną jako „best”. Jest to wartość kryterium osobnika który nie spełnia wszystkich kryteriów najlepiej, ale ma najniższą wartość jednego konkretnego kryterium.

Operator ERX daje częściowo lepsze rezultaty niż AEX, ale różnica nie jest kolosalna. PMX deklasuje konkurencję pod każdym względem. Potrafi znaleźć rozwiązanie które dobrze spełnia wszystkie trzy kryteria, oraz rozwiązania gdzie pojedyncze kryteria zostały dobrze poprawione, a wszystko to w krótszym czasie. Być może istnieje możliwość poprawy efektywności operatorów AEX oraz ERX, ale należy zastanowić się, czy jest to warte czasu. Dorównanie efektywności operatora PMX może być trudne.

	Operator		
	AEX	ERX	PMX
Avg Impr	13,393%	15,037%	26,417%
Pop Impr	-2,133%	-0,066%	5,085%
Weight Impr	1,941%	0,179%	4,018%
Dist Impr	40,372%	44,999%	70,149%
Best Pop Impr	6,137%	6,619%	14,545%
Best Weight Impr	6,449%	6,451%	15,467%
Best Dist Impr	40,372%	44,999%	70,149%

Tabela 4.15: Porównanie operatorów krzyżowania.

Finałowe wnioski są jednoznaczne - nie ma powodów stosowania operatorów AEX oraz ERX w takiej formie, PMX jest o wiele bardziej efektywny i wydajny.

5 Analiza wyników i wnioski

Zaprojektowany algorytm genetyczny poprawnie rozwiązuje problem optymalizacji układu towaru w magazynie wykorzystując do tego trzy kryteria oceny. Jak wskazują badania parametry wejściowe mają faktyczny wpływ na jego efektywność jak i czas działania. Przy poprawnej konfiguracji efektywność algorytmu jest zadowalająca. Operator PMX daje zdecydowanie najlepsze rezultaty, średnia poprawa wynosi około 25%, a dla operatorów AEX i ERX około 14%, aczkolwiek operator ERX daje częściej lepszy wynik niż AEX. Trzeba jednak przyznać, że temat nie został w pełni wyczerpany, i pewne elementy można spróbować zbadać w przyszłości.

Aby zwiększyć praktyczność proponowanego rozwiązania, należy dodać mechanizm który pozwala na używanie danych gdzie identyfikator towaru to ciąg znaków, a nie liczba całkowita. W praktyce do identyfikowania towarów najczęściej używa się ciągu liter i cyfr, i dostosowanie programu do obsługi takich danych byłoby wskazane.

Warto zbadać również inne operatory krzyżowania, albo nawet spróbować stworzyć nowe (od zera lub poprzez modyfikację istniejących), gdyż być może będą one bardziej efektywne. W szczególności należałoby skupić się na operatorach które nie interpretują genotypu jako ciąg krawędzi lub łuków. AEX oraz ERX interpretują genotyp jako takie ciągi, i okazały się one mało efektywne w szukaniu rozwiązań które dobrze spełniają wszystkie kryteria w tym konkretnym zagadnieniu.

Duży wpływ na efektywność algorytmu może mieć dodanie przeszukiwania lokalnego do operatorów krzyżowania. Mechanizm ten jest możliwy do wprowadzenia do operatora AEX - operator ten gdy nie może znaleźć odpowiedniego elementu genotypu wybiera element losowy (który jednak spełnia pewne wymagania). Losowość można zastąpić przeszukiwaniem lokalnym.

W praktyce nie każdy układ towaru jest możliwy, należy uwzględnić środek ciężkości regału. Jeżeli towar nie jest ułożony odpowiednio równomiernie względem wagi, regał może być niestabilny. Problem ten można rozwiązać na wiele sposobów. Można spróbować dodać kolejne kryterium oceny, wprowadzić funkcję kary, albo próbować zminimalizować problemy przeszukiwaniem lokalnym.

Należałoby również sprawdzić, czy istnieje możliwość zastosowania innych operatorów selekcji niż selekcja turniejowa. Selekcja ruletki jest bardzo często efektywna, i można by spróbować dokonać jej modyfikacji, aby była kompatybilna z algorytmem wielokryterialnym.

W przypadku mutacji ciekawym testem byłoby użycie mutacji dynamicznej, gdzie prawdopodobieństwo i siła mutacji zmienia się w zależności od pewnych parametrów, na przykład od wskaźnika różnorodności populacji. Z kolei tutaj można by sprawdzić, czy różne metody wyznaczania tego wskaźnika również wpływają na wynik.

Sensownie również byłoby spróbować ulepszyć inicjalizację populacji. Losowe generowanie nie jest nieakceptowalne, ale o wiele lepiej spróbować pokryć przestrzeń poszukiwań w sposób bardziej inteligentny.

Spis algorytmów

3.1	Fitness	37
3.2	Wyznaczanie rangi	38
3.3	Wyznaczanie odległości zagęszczenia	39
3.4	Selekcja turniejowa	40
3.5	Mutacja	41
3.6	Operator AEX	42
3.7	Operator ERX	42
3.8	Operator PMX	43
3.9	Wyznaczanie znormalizowanej sumy	44
3.10	Wyznaczanie średniej poprawy	45

Spis rysunków

1.1	Schemat działania algorytmu genetycznego.	9
1.2	Dominacja Pareto - wizualizacja.	14
1.3	Odległość zagęszczenia - wizualizacja.	16
2.1	Kodowanie osobnika.	24
3.1	Diagram kompletnego programu.	31
3.2	Diagram algorytmu genetycznego.	33
3.3	Diagram tworzenia nowej populacji.	34
3.4	Interfejs użytkownika - lewy panel.	47
3.5	Interfejs użytkownika - prawy panel.	48
3.6	Interfejs użytkownika podczas działania algorytmu.	49
3.7	Testy jednostkowe.	51
3.8	Okno programu dotMemory - użycie pamięci RAM dla AEX.	53
3.9	Okno programu dotTrace - lewy panel - użycie procesora dla operatora AEX.	54
3.10	Okno programu dotTrace - prawy panel - użycie procesora dla operatora AEX.	54
4.1	Przebieg algorytmu dla dużego magazynu dla operatora AEX.	66
4.2	Przebieg algorytmu dla dużego magazynu dla operatora ERX.	73
4.3	Przebieg algorytmu dla dużego magazynu dla operatora PMX.	79

Spis tabel

1.1	Tabela porównania algorytmów - model.	18
1.2	Tabela porównania algorytmów - operatory.	19

3.1	Użycie pamięci RAM.	53
3.2	Porównanie metod pod wpływem użycia procesora dla operatora AEX.	55
3.3	Porównanie metod pod wpływem użycia procesora dla operatora ERX.	56
3.4	Porównanie metod pod wpływem użycia procesora dla operatora PMX.	56
4.1	Wpływ rozmiaru populacji dla AEX.	59
4.2	Wpływ prawdopodobieństwa mutacji dla AEX.	61
4.3	Wpływ ilości mutowanych osobników dla AEX.	62
4.4	Wpływ liczby podmian elementów genotypu dla AEX.	63
4.5	Wpływ liczby rodziców dla AEX.	64
4.6	Wpływ rozmiaru populacji dla ERX.	67
4.7	Wpływ prawdopodobieństwa mutacji dla ERX.	68
4.8	Wpływ ilości mutowanych osobników dla ERX.	69
4.9	Wpływ liczby podmian elementów genotypu dla ERX.	70
4.10	Wpływ liczby rodziców dla ERX.	71
4.11	Wpływ rozmiaru populacji dla PMX.	74
4.12	Wpływ prawdopodobieństwa mutacji dla PMX.	75
4.13	Wpływ ilości mutowanych osobników dla PMX.	76
4.14	Wpływ liczby podmian elementów genotypu dla PMX.	77
4.15	Porównanie operatorów krzyżowania.	80

Bibliografia

- [1] Abraham A., Jain L., Goldberg R., *Evolutionary Multiobjective Optimization: Theoretical Advances and Applications*. Springer, 2005.
- [2] Ahmed Z. H., *A Comparative Study of Eight Crossover Operators for the Maximum Scatter Travelling Salesman Problem*. International Journal of Advanced Computer Science and Applications, t. 11, nr. 6, 2020.
- [3] Avdekins A., Savrasovs M., *Making warehouse logistics smart by effective placement strategy based on genetic algorithms*. Transport and Telecommunication, t. 20, nr. 4, s. 318–324, 2019.
- [4] Branke J., Deb K., Miettinen K., Słowiński R., *Multiobjective Optimization: Interactive and Evolutionary Approaches*. Springer, 2008.
- [5] Cheng C.-B., Weng Y.-C., *Warehouse Storage Assignment by Genetic Algorithm with Multi-objectives*. Proceedings of the International Conference on Industrial Engineering and Operations Management, 2018.
- [6] Coello C. A. C., Lamont G. B., Veldhuizen D. A. V., *Evolutionary Algorithms for Solving Multi-Objective Problems: Second Edition*. Springer, 2007.
- [7] Deb K., *Multi-Objective Optimization using Evolutionary Algorithms*. John Wiley & Sons, 2001.
- [8] Filho J. R., Alippi C., Treleaven P., *Genetic Algorithm Programming Environments*. Computer, t. 27, nr. 6, s. 28–43, 1994.
- [9] Haupt R. L., Haupt S. E., *Practical Genetic Algorithms Second Edition*. John Wiley & Sons, 2004.
- [10] Koza J. R., *Genetic Programming: On the Programming of Computers by Means of Natural Selection*. MIT Press, 1992.
- [11] Kramer O., *Genetic Algorithm Essentials*. Springer, 2017.
- [12] Mitchell M., *An Introduction to Genetic Algorithms*. MIT Press, 1998.
- [13] Pal S. K., Wang P. P., *Genetic Algorithm for Pattern Recognition*. CRC Press, 1996.
- [14] Puljić K., Manger R., *Comparison of eight evolutionary crossover operators for the vehicle routing problem*. Mathematical Communications, t. 18, nr. 2, s. 359–375, 2013.
- [15] Qin G., Li J., Jiang N., Li Q., Wang L., *Warehouse Optimization Model Based on Genetic Algorithm*. Mathematical Problems in Engineering, t. 2013, 2013.

- [16] Reeves C. R., Rowe J. E., *Genetic Algorithms: Principles and Perspectives - A Guide to GA Theory*. Kluwer Academic Publishers, 2002.
- [17] Sheppard C., *Genetic Algorithms with Python*. 2016.
- [18] Sivanandam S., Deepa S., *Introduction to Genetic Algorithms*. Springer, 2008.
- [19] Smith S., Southerby M., Setiniyaz S., Apsimon R., Burt G., *Multiobjective optimization and Pareto front visualization techniques applied to normal conducting rf accelerating structures*. Physical Review Accelerators and Beams, t. 25, nr. 6, 2022.
- [20] Sun J., Deng J., Li Y., *Indicator & crowding Distance-Based Evolutionary Algorithm for Combined Heat and Power Economic Emission Dispatch*. Applied Soft Computing, t. 90, 2020.
- [21] Tan K., Khor E., Lee T., *Multiobjective Evolutionary Algorithms and Applications*. Springer, 2005.
- [22] Wang W., Gao J., Gao T., Zhao H., *Optimization of Automated Warehouse Location Based on Genetic Algorithm*. Advances in Intelligent Systems Research, t. 134, 2017.
- [23] Whitley D., *A Genetic Algorithm Tutorial*. Statistics and Computing, t. 4, s. 65–85, 1994.